



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA IN INFORMATICA

CASO DI STUDIO
PER IL CORSO DI
INGEGNERIA DELLA CONOSCENZA

PokéVision: Ontologia e Card Recommender per Pokémon TCG

STUDENTI:

Giulia Panfilò - 755423 - g.panfilo1@studenti.uniba.it

Gianni Navolio - 758412 - g.navolio@studenti.uniba.it

Indice

1	Introduzione	1
2	Regolamento di Pokémon TCG	2
2.1	Struttura generale di una carta	2
2.2	Energie	3
2.3	Carte Trainer	4
2.4	Carte Pokémon	5
2.5	Costruzione di un mazzo standard	8
2.6	Turno e zone di gioco	9
2.6.1	Campo di gioco del Pokémon TCG	9
2.6.2	Predisposizione della partita	10
2.6.3	Il turno e i suoi passaggi	10
2.6.4	Condizioni di vittoria	11
3	Ontologia	12
3.1	Modellazione del dominio	12
3.2	Integrazione con BGO e riferimenti allo stato dell'arte	15
3.3	Definizione dell'ontologia per concetti	15
3.3.1	Concetti legati alla Carta (<code>pkmnTCG:Card</code>)	15
3.3.2	Concetto Carta Pokémon (<code>pkmnTCG:PokemonCard</code>)	16
3.3.3	Concetto Allenatore (<code>pkmnTCG:TrainerCard</code>)	17
3.3.4	Concetto Energia (<code>pkmnTCG:Energy</code>)	17
3.3.5	Concetti di Attacco (<code>pkmnTCG:Attack</code>)	18
3.3.6	Concetti legati al Mazzo (<code>pkmnTCG:Deck</code>)	18
4	Dati e strumenti	20
4.1	Primi tentativi	20
4.1.1	Limiti riscontrati	20
4.1.2	Cambio di approccio	21
4.2	Dataset principali	21
4.3	Struttura dei dati e selezione delle features	21

4.3.1	Carte	22
4.3.2	Deck	23
4.3.3	Set	24
4.4	Rappresentazioni dei Deck	24
5	Knowledge Base	26
5.1	Creazione dei Fatti	28
5.1.1	Serializzare una Carta in RDF	28
5.1.2	Serializzare un Deck in RDF	29
5.2	Validazione dei Fatti	30
5.2.1	Pattern di query SPARQL	30
5.2.2	Validazione con pySHACL	39
5.3	Regole e inferenza	41
5.3.1	SWRL (deduzioni strutturali)	41
5.3.2	Vincoli numerici: strategia	43
5.4	Esecuzione (Python, Owlready2).	44
6	Card Recommender	45
6.1	Metriche di similarità e differenza	45
6.2	Clustering dei mazzi	47
6.2.1	Metodi di clustering	47
6.2.2	Determinazione del numero ottimale di cluster	48
6.3	Valutazione delle rappresentazioni	55
6.3.1	Metriche di valutazione	55
6.4	Masked Prediction	56
6.4.1	Come funziona	56
6.4.2	Risultati	57
6.4.3	Vantaggi e limiti	58
6.4.4	Sviluppi futuri	58
7	Estratti	60
	Bibliografia	68

Capitolo 1

Introduzione

I giochi di carte collezionabili (Trading Card Games, TCG) rappresentano una delle forme più diffuse di intrattenimento moderno. La loro particolarità risiede nella combinazione tra collezione e competizione: ogni giocatore costruisce un mazzo unico selezionando carte da un insieme più ampio, caratterizzato sia da regole comuni che da infinite possibilità di combinazione.

L'esperienza di gioco si articola infatti in due momenti distinti ma complementari: da un lato la costruzione del mazzo (deckbuilding), fase creativa e analitica in cui si scelgono le carte più adatte a una strategia; dall'altro l'utilizzo del mazzo in partita, dove si mette alla prova la propria abilità decisionale, l'adattamento alle mosse avversarie e la capacità di sfruttare le sinergie pianificate.

Tra i TCG più conosciuti spicca Pokémon, brand nato come videogioco negli anni '90 e presto declinato in serie animate, fumetti e prodotti di intrattenimento di ogni genere. Il Pokémon Trading Card Game (Pokémon TCG) ne costituisce una delle forme più longeve e popolari: un gioco competitivo che trasporta l'universo dei Pocket Monsters nel formato delle carte, permettendo ai giocatori di impersonare allenatori e di utilizzare Pokémon, Energie e Strumenti per affrontarsi in duelli. Col tempo, il Pokémon TCG si è imposto come fenomeno globale, con tornei ufficiali, circuiti professionali e una comunità internazionale di appassionati.

All'interno di questo contesto nasce **PokéVision**, un sistema esperto pensato per supportare la costruzione di mazzi da parte di giocatori, in particolare di chi è meno esperto. PokéVision si basa sull'analisi di dataset torneistici, ovvero collezioni di deck usati in competizioni ufficiali, dai quali estrarre pattern, strategie e scelte ricorrenti. Grazie a questo patrimonio di conoscenza, il sistema è in grado di proporre aggiunte mirate per completare un deck incompleto;

L'idea di PokéVision nasce dall'esigenza personale, condivisa da molti giocatori che non considerano il deckbuilding una parte stimolante dell'esperienza di gioco, di avere uno strumento che semplifichi le scelte nella costruzione di un deck.

Capitolo 2

Regolamento di Pokémon TCG

Il **Gioco di Carte Collezionabili Pokémon** (Pokémon TCG) è un gioco strategico a turni in cui due giocatori, nel formato che andremo a trattare per i fini di questo progetto, si sfidano impersonando Allenatori e utilizzando squadre di Pokémon, rappresentati dalle carte. Ogni mazzo è composto da 60 carte e combina diversi tipi di elementi: i *Pokémon* che combattono sul campo, le *Energie* che permettono di attaccare e le *Carte Allenatore* che forniscono supporto o modificano l'andamento della partita. In questa sezione verranno descritti i principali tipi di carte e il loro ruolo nel sistema di gioco.

2.1 Struttura generale di una carta

Tutte le carte nel gioco, indipendentemente dal tipo, condividono alcuni attributi fondamentali che ne permettono l'identificazione e l'utilizzo. Questi elementi costituiscono la base alla quale si aggiungono le informazioni specifiche di ciascun supertipo (Pokémon, Energia, Allenatore).

Attributo	Descrizione
Nome	Nome della carta.
Categoria	Supertype della carta (Pokémon, Energia, Allenatore).
Codice di espansione	Sigla che identifica il set di appartenenza.
Numero di collezione	Posizione della carta all'interno del set.
Illustrazione	Parte grafica che caratterizza la carta.
Testo	Effetti o regole specifiche applicabili in gioco.

Tabella 2.1: Attributi comuni a tutte le carte del Pokémon TCG.

2.2 Energie

Le carte **Energia** forniscono la possibilità ai Pokémon di effettuare alcune azioni di gioco, tra cui gli attacchi. Le *Energie Base* (es. Erba, Fuoco, Acqua, Lampo, Psico, Lotta, Oscurità, Metallo) non hanno limite di copie nel mazzo.



Figura 2.1: Energie base in Pokémon TCG

Le *Energie Speciali*, invece, seguono la regola generale delle 4 copie per nome.



Figura 2.2: Esempio di Energia Speciale in Pokémon TCG

2.3 Carte Trainer

Le **Carte Allenatore** modulano ritmo e consistenza del mazzo: cercano carte, ripristinano risorse, alterano il campo o inibiscono l'avversario. Si distinguono in *Strumento*, *Aiuto*, *Stadio* e *Oggetto Pokémon*. In generale, si possono giocare più carte Allenatore per turno ma solo *una* carta Aiuto e *una* carta Stadio per turno.



Figura 2.3: Esempio di Carta Allenatore di tipo Strumento.

2.4 Carte Pokémon

Le carte **Pokémon** rappresentano le entità principali del gioco e contengono tutte le informazioni necessarie a definire caratteristiche, evoluzioni e azioni disponibili. Si possono giocare un numero qualsiasi di carte Pokémon in un turno, fino a riempire la panchina. Ogni carta è composta da più sezioni, illustrate di seguito.



Figura 2.4: Esempio di carta Pokémon con le principali sezioni evidenziate.

Tipo e Punti Salute

Il *tipo* di un Pokémon determina il tipo dei suoi attacchi. I *Punti Salute* (PS), indicati in alto a destra, stabiliscono quanti danni può sopportare prima di essere messo KO.

Evoluzione

Ogni Pokémon appartiene a una linea evolutiva che può comprendere tre stadi: Base, Fase 1 e Fase 2.

Solo i Pokémon Base possono essere messi direttamente in campo. Si può giocare un pokémon non Base se sul campo è presente da almeno un turno un pokémon della



Figura 2.5: Esempio di linea evolutiva di Sprigatito.

stessa linea evolutiva di fase immediatamente precedente, posizionandolo direttamente sopra.

Attacchi

Ogni carta riporta uno o più *attacchi*, con relativo costo in Energie, danno inflitto e possibili effetti aggiuntivi (come condizioni speciali o spostare Energie in gioco).

Abilità

Alcuni Pokémon possiedono un'*abilità*, cioè un effetto speciale che non richiede Energie. Possono essere **effetti statici** sempre attivi (es. "Questo Pokémon subisce 30 danni in meno dagli attacchi."), oppure **effetti attivabili** una volta per turno o al verificarsi di una condizione. L'uso di un'abilità non impedisce di attaccare nello stesso turno.

Metadati della carta

Ogni carta Pokémon riporta un *codice di espansione* e un *numero di collezione*, utili a distinguere le carte anche quando hanno lo stesso nome ma provengono da set diversi.

Debolezze e Resistenze

Sono meccaniche di gioco che modificano il danno che un Pokémon difensore subisce da un attaccante. Sono indicate da un'icona dell'energia e dalla modifica al danno, chiamata 'ammontare'. Un valore tipico di debolezza è $\times 2$, ma questi possono cambiare, e anziché essere moltiplicatori, sottrarre o aggiungere un ammontare fisso di danno.

La maggior parte dei casi presenta una resistenza e una debolezza, ma esistono



Figura 2.6: Esempio di Pokémon dotato di abilità.

Pokémon (per lo più di tipo drago o di sottotipo 'ex') che presentano nessuna o due resistenze.

Nel progetto, per semplicità, debolezze e resistenze sono state trattate come valori booleani senza gestire i modificatori numerici.

Costo di ritirata

Quantità e tipo di energie che è necessario scartare dal pokémon attivo per sostituirlo con uno dei pokémon in panchina.

Elemento	Descrizione
Tipo	Determina il tipo degli attacchi del Pokémon.
Punti Salute	Quantità di danni che il Pokémon può subire prima del KO.
Debolezze	Tipi da cui il Pokémon subisce danni aumentati.
Resistenze	Tipi da cui il Pokémon subisce danni ridotti.
Fase Evolutiva	Stadio nella linea Base $\rightarrow$ Fase 1 $\rightarrow$ Fase 2.
Attacchi	Azioni disponibili, con costo in Energie e eventuale danno.
Abilità	Effetti speciali permanenti o attivabili senza Energie.
Codice di espansione	Sigla che identifica il set di appartenenza.
Numero di collezione	Posizione della carta all'interno del set.
Costo di ritirata	Energie da scartare per sostituire.

Tabella 2.2: Componenti principali di una carta Pokémon.

2.5 Costruzione di un mazzo standard

Un mazzo del **Pokémon TCG** deve rispettare alcune regole precise per poter essere considerato legale in un torneo:

- Deve contenere **esattamente 60 carte**.
- Deve includere **almeno un Pokémon Base**, necessario per iniziare la partita.
- Deve contenere al massimo **quattro copie di carte con lo stesso nome**, eccezion fatta per le *Energie Base*.

Oltre a questi vincoli regolamentari, la costruzione di un mazzo competitivo implica considerazioni strategiche legate al bilanciamento delle categorie di carte. In generale, un mazzo efficace prevede una distribuzione equilibrata tra *Pokémon*, *Carte Allenatore* ed *Energie*. La quantità di Energie dipende dal tipo di strategia adottata: in media si utilizzano tra le 12 e le 15 Energie, solitamente di uno o due tipi per pagare più facilmente i costi degli attacchi. I Pokémon di tipo Incolore, che possono sfruttare qualunque Energia, vengono spesso scelti per aumentare la flessibilità del mazzo.

Le Carte Allenatore costituiscono una parte fondamentale della costruzione, poiché permettono di cercare Pokémon, pescare nuove carte, ripristinare risorse o limitare le mosse avversarie. È consigliabile includerne tra le 20 e le 25, con una combinazione di Strumenti, Aiuti e Stadi.

Il resto delle carte è costituito dai Pokémon stessi, che formano il nucleo della strategia: oltre a garantire una linea evolutiva stabile, è buona pratica inserire più copie di Pokémon chiave per ridurre il rischio di non poterli giocare quando necessario.

Elemento	Tipo di vincolo	Descrizione
Numero di carte	Rigido	Esattamente 60 carte per mazzo.
Pokémon Base	Rigido	Almeno 1 Pokémon Base obbligatorio.
Copie massime	Rigido	4 copie per carta con lo stesso nome, eccetto Energie Base
Energie	Flessibile	12–15, preferibilmente di uno o due tipi per coerenza.
Carte Allenatore	Flessibile	20–25, per garantire consistenza e controllo.
Pokémon	Flessibile	Il resto del mazzo, con più copie dei Pokémon chiave.

Tabella 2.3: Vincoli rigidi e linee guida flessibili nella costruzione di un mazzo standard del gioco Pokémon TCG.

2.6 Turno e zone di gioco

2.6.1 Campo di gioco del Pokémon TCG

Il campo di gioco si divide in zone, ognuna designata a carte specifiche.

- **Posizione attiva:** la zona in cui si trova il Pokémon attivo, che può attaccare o essere sostituito. In questa zona deve sempre esserci un Pokémon già dall'inizio della partita infatti viene posizionato qui un Pokémon Base.
- **Panchina:** zona in cui si trovano gli altri Pokémon in gioco. Possono essere sostituiti al Pokémon attivo pagando il costo di ritirata ad esso associato. I Pokémon in panchina non possono attaccare, ma possono attivare **Abilità**.
- **Pila degli scarti:** tutte le carte rimosse dalla partita, tra cui le carte allenatore utilizzate durante il turno, le energie rimosse e i Pokémon che esauriscono i propri HP, vengono posizionati in questa zona e non possono più essere utilizzati.
- **Carte Premio:** sei carte coperte scelte casualmente dal mazzo all'inizio della partita. Ogni volta che un giocatore manda KO il Pokémon di un avversario, pesca una carta da questa zona. Un giocatore vince quando pesca l'ultima carta premio, o prima se l'avversario perde.



Figura 2.7: Zone di gioco

2.6.2 Predisposizione della partita

All'inizio della partita si pone il proprio deck nella zona **Mazzo** e si pescano le prime sette carte come mano iniziale. Qualora non fosse presente almeno un Pokémon Base tra queste, la mano va mostrata all'avversario e reinserita nel mazzo per sostituirla con altre sette carte casuali. Ogni volta che si effettua questa sostituzione l'avversario pesca una carta aggiuntiva.

Quando entrambi hanno la mano iniziale definitiva, si pescano le prime sei carte del mazzo e si dispongono coperte nella zona **Carte premio**.

Il lancio di una moneta determina il primo giocatore. Ogni giocatore sceglie il Pokémon Base da giocare a faccia in giù in **Posizione Attiva** e al massimo altri cinque da disporre coperti nella zona **Panchina**.

2.6.3 Il turno e i suoi passaggi

Ciascun turno di un giocatore si compone di tre fasi:

1. inizialmente pesca una carta,
2. poi sceglie quali azioni di gioco effettuare tra le seguenti, nell'ordine che preferisce:

- Sceglie qualsiasi numero di Pokémon Base dalla mano e li posiziona in zona Panchina;
- Fa evolvere i Pokémon che sono in campo da almeno un turno;
- Assegna una sola carta Energia tra quelle che ha in mano a uno solo dei Pokémon in campo;
- Gioca carte Alleatore (tutte le carte Strumento che vuole, ma solo una carta Stadium e una carta Aiuto per turno). Il giocatore che inizia per primo non può giocare carte Aiuto durante il primo turno;
- Ritira il Pokémon attivo scartando energie assegnatagli pari al suo costo di ritirata e lo sostituisce con uno di quelli in panchina;
- Attiva le abilità dei Pokémon in campo.

3. infine attacca e termina il turno.

2.6.4 Condizioni di vittoria

La partita si conclude con la vittoria di un giocatore se

- ha pescato tutte le sue carte premio;
- l'avversario non può pescare;
- l'avversario non ha più Pokémon in gioco.

Se una di queste condizioni si realizza, il giocatore ha vinto la partita.

Capitolo 3

Ontologia

In questo capitolo viene esposta la struttura e la logica dell'ontologia sviluppata per il progetto.

Nel contesto dell'ingegneria della conoscenza, un'ontologia è una formalizzazione esplicita delle entità di un dominio, delle loro proprietà e delle relazioni che le connettono. Serve a fornire una rappresentazione formale, condivisa e leggibile da una macchina, che consenta a sistemi software di ragionare sui dati, di verificarne la coerenza e di inferire nuove conoscenze. Il suo ruolo è cruciale per la gestione automatizzata della complessità del dominio in esame.

3.1 Modellazione del dominio

La modellazione del dominio inizialmente è stata orientata in modo generale verso l'universo ludico di Pokémon, considerando la possibilità di adattare un'ontologia esistente. Tuttavia, è emerso che le differenze tra le dinamiche digitali del videogioco e le meccaniche fisiche del TCG rendevano tale approccio impraticabile, in quanto l'ontologia preesistente era ricca di classi concettualmente distanti e proprietà non pertinenti al nostro scopo.

Abbiamo quindi deciso di adottare invece un **approccio misto** e di progettare la nostra ontologia partendo dai concetti più importanti per un gioco di carte: il **mazzo** (Deck) e la **carta** (Card). Abbiamo definito poi tutti gli altri concetti e le loro relazioni, come i tipi di carte, gli attacchi, le abilità, le zone di gioco, eccetera.

- **Struttura delle Carte:** La classe `pkmnTCG:Card` è il punto di partenza per sottoclassi specializzate che rappresentano i diversi tipi di carte presenti nel gioco: `pkmnTCG:PokemonCard`, `pkmnTCG:EnergyCard` e `pkmnTCG:TrainerCard`.
- **Struttura dei Mazzi:** L'idea iniziale era di rappresentare un mazzo direttamente come una lista di carte, ma durante la progettazione e stesura dei

vincoli, è risultato necessario definire la classe `pkmnTCG:DeckEntry`. Questa è un'entità intermedia che collega il mazzo alle carte uniche che contiene, specificando per ciascuna la quantità di copie presenti, superando così i limiti di una semplice lista di occorrenze. Un mazzo è quindi composto da istanze di `DeckEntry`, ognuna delle quali è definita dalle proprietà `pkmnTCG:entryCard`, che ne specifica l'istanza di carta, e `pkmnTCG:entryQuantity`, che ne determina il numero di copie.

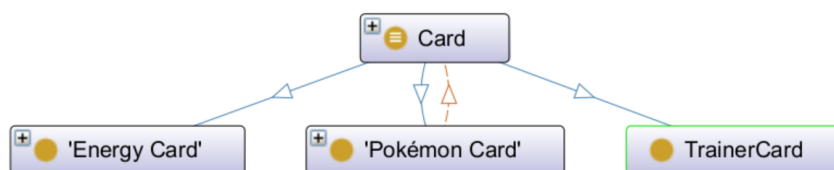


Figura 3.1: Grafico di specializzazione sottoclassi Card

- **Gestione dell'Energia e dei Costi:** Per formalizzare l'uso dell'energia, sono state definite le classi `pkmnTCG:EnergyCard` e `pkmnTCG:Energy`, ossia la carta di supertipo "Energia", e l'energia stessa che questa rappresenta. Per rappresentare il costo di determinate azioni come attacchi e il ritiro di un pokemon, abbiamo introdotto la classe `pkmnTCG:Cost`, che ne specifica la quantità e il tipo di energia necessari. Il set di energie possibili (es. `pkmnTCG:Fire`, `pkmnTCG:Water`, ...) è stato definito in modo esaustivo usando l'assioma `owl:oneOf` sulla classe `pkmnTCG:Energy`.

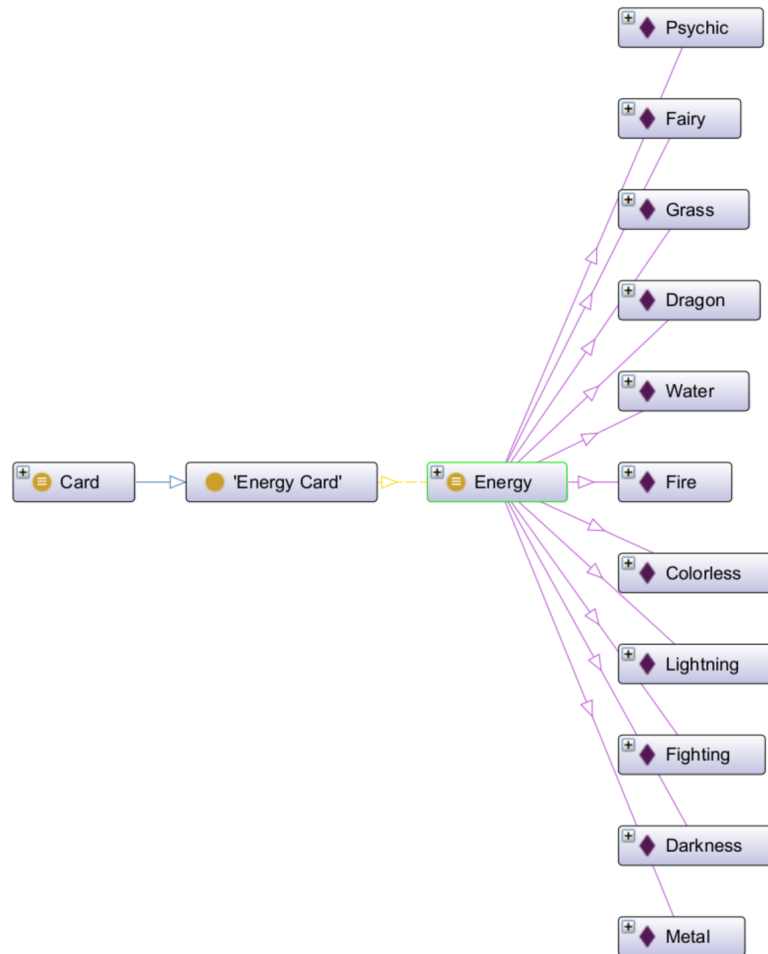


Figura 3.2: Rappresentazione grafo individui Energy

- **Concetti e Stati di Gioco:** Il TCG è composto da regole complesse che richiedono una modellazione specifica. Per modellare questi aspetti, sono state create classi e proprietà apposite:
- **Zone di Gioco:** La classe `pkmnTCG:Zone` è stata istanziata in individui che rappresentano le diverse aree di gioco. Abbiamo usufruito dell'assioma `owl:AllDisjointClasses` in modo da definire che gli individui sono reciprocamente esclusivi, riflettendo la realtà fisica del tabellone di gioco.
- **Entità di Gioco:** Quali `pkmnTCG:Player`, `pkmnTCG:Turn`, `pkmnTCG:Action` e le loro sottoclassi.

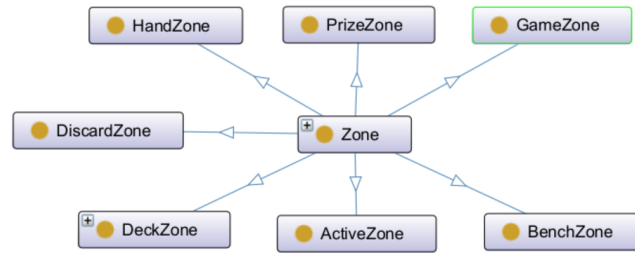


Figura 3.3: Rappresentazione grafica individui Zone

3.2 Integrazione con BGO e riferimenti allo stato dell'arte

L'ontologia applicativa sviluppata per il Pokémon TCG non è stata progettata in modo isolato, ma si è basata su modelli e tassonomie esistenti nel dominio dei giochi. In particolare, è stata referenziata la **Board Game Ontology** (BGO), che fornisce un vocabolario generale per i giochi da tavolo comprendente entità come **Player** e **Card**. L'integrazione di BGO ha permesso di radicare la nostra modellazione su una base solida, sfruttandone le astrazioni per descrivere le entità fisiche e i metadati delle carte.

Inoltre sono state utili a scopo progettuale, ma senza riferimenti diretti per mancanza di ontologie formali, il **Game Ontology Project** (GOP), ossia una tassonomia di riferimento nei *game studies* per la descrizione multilivello dei giochi, che distingue regole dinamiche da elementi esperienziali, oltre che la **Game Mechanics Ontology** (GMO) proposta da Kritz et al[4] che riorganizza criticamente la lista di meccaniche di BoardGameGeek attraverso il framework **MDA (Mechanics–Dynamics–Aesthetics)**, spunto metodologico rilevante per distinguere regole implementabili, comportamenti emergenti ed esperienze di gioco.

3.3 Definizione dell'ontologia per concetti

L'ontologia del Pokémon TCG è stata definita formalmente in OWL e serializzata nel formato Turtle (`.ttl`). Questa sezione riorganizza la sua struttura per concetto, descrivendo ogni elemento (classe, proprietà e individui) in modo integrato.

3.3.1 Concetti legati alla Carta (`pkmnTCG:Card`)

La classe `pkmnTCG:Card` è la radice gerarchica per tutte le carte del Pokémon TCG. Eredita le proprietà dal vocabolario generico `bgo:Card` e si specializza in sottoclassi.

Di seguito le proprietà della classe `pkmnTCG:Card` presentate in formato tabellare, con i rispettivi **range** e cardinalità.

Nome	Range	Cardinalità
<code>:hasID</code>	<code>xsd:string</code>	1
<code>:hasName</code>	<code>rdfs:Literal</code>	1
<code>:hasCollectionNumber</code>	<code>xsd:integer</code>	1
<code>:hasDescription</code>	<code>xsd:string</code>	any
<code>:hasRule</code>	<code>xsd:string</code>	any
<code>:hasSubType</code>	<code>xsd:string</code>	any
<code>:hasSet</code>	<code>:Set</code>	1
<code>:inZone</code>	<code>:Zone</code>	1
<code>:represents</code>	<code>:Pokemon</code> <code>:Energy</code>	1
<code>:playedOnTurn</code>	<code>:Turn</code>	1

Tabella 3.1: I predicati con cardinalità 1 sono `owl:FunctionalProperty`.

3.3.2 Concetto Carta Pokémon (`pkmnTCG:PokemonCard`)

La classe `pkmnTCG:PokemonCard` è una sottoclasse di `pkmnTCG:Card`. Rappresenta il concetto centrale di un Pokémon giocabile. Di seguito le proprietà della classe `pkmnTCG:PokemonCard` presentate in formato tabellare, con i rispettivi **range** e cardinalità.

Nome	Range	Cardinalità
:hasDamageCounters	xsd:nonNegativeInteger	any
:hasAttack	:Attack	any
:hasAbility	:Ability	any
:hasResistanceTo	:Energy	any
:hasWeaknessTo	:Energy	any
:hasAttachedCard	:Card	any
:hasEnergy	:Energy	any
:hasStatus	:Status	any
:hasRetreatCost	:Cost	1
:hasHP	xsd:nonNegativeInteger	1

Tabella 3.2: I predicati con cardinalità 1 sono owl:FunctionalProperty.

3.3.3 Concetto Allenatore (pkmnTCG:TrainerCard)

La classe `pkmnTCG:TrainerCard` è una sottoclasse di `pkmnTCG:Card`. Di seguito le proprietà della classe `pkmnTCG:TrainerCard` presentate in formato tabellare, con i rispettivi `range` e cardinalità.

Nome	Range	Cardinalità
:hasEffect	xsd:string	any

3.3.4 Concetto Energia (pkmnTCG:Energy)

La classe `pkmnTCG:Energy` è stata definita come una classe enumerativa che specifica gli unici individui che possono appartenerle.

Individui della classe `pkmnTCG:Energy`

- Gli unici individui ammessi sono: `pkmnTCG:Colorless`, `pkmnTCG:Darkness`, `pkmnTCG:Dragon`, `pkmnTCG:Fairy`, `pkmnTCG:Fighting`, `pkmnTCG:Fire`, `pkmnTCG:Grass`, `pkmnTCG:Lightning`, `pkmnTCG:Metal`, `pkmnTCG:Psychic` e `pkmnTCG:Water`.

- L'assioma `owl:oneOf` garantisce l'integrità dei dati, impedendo la creazione di tipi di energia non validi.
- Questi individui sono stati anche classificati come `skos:Concept` e collegati a `pkmnTCG:EnergyConceptScheme` per una gestione strutturata.

3.3.5 Concetti di Attacco (`pkmnTCG:Attack`)

La classe `pkmnTCG:Attack` descrive le capacità offensive di un Pokémon. Di seguito le proprietà della classe `pkmnTCG:Attack` presentate in formato tabellare, con i rispettivi `range` e cardinalità.

Nome	Range	Cardinalità
<code>:attackName</code>	<code>xsd:string</code>	1
<code>:attackDamage</code>	<code>xsd:int</code>	any
<code>:attackEffect</code>	<code>xsd:string</code>	any
<code>:attackCost</code>	<code>:Cost</code>	1
<code>:targetsPokemon</code>	<code>:PokemonCard</code>	any

Tabella 3.3: I predicati con cardinalità 1 sono `owl:FunctionalProperty`.

3.3.6 Concetti legati al Mazzo (`pkmnTCG:Deck`)

La classe `pkmnTCG:Deck` rappresenta un mazzo di gioco. La sua composizione è modellata attraverso la classe intermedia `pkmnTCG:DeckEntry`. Di seguito le proprietà della classe `pkmnTCG:Deck` presentate in formato tabellare, con i rispettivi `range` e cardinalità.

Nome	Range	Cardinalità
<code>:hasDeckEntry</code>	<code>:DeckEntry</code>	any
<code>:hasArchetype</code>	<code>xsd:string</code>	any

Concetti di Entrata del Mazzo (`pkmnTCG:DeckEntry`)

Questa classe intermedia specifica la quantità e la carta di un'entrata nel mazzo.

Nome	Range	Cardinalità
:entryCard	:Card	1
:entryQuantity	xsd:positiveInteger	1

Capitolo 4

Dati e strumenti

4.1 Primi tentativi

La raccolta dei dati necessari allo sviluppo del sistema è avvenuta in più fasi, tramite diverse fonti. Inizialmente, si è scelto di partire dall'[API di Pokémon](#)[3] e scaricare le informazioni strutturate sulle carte: immagini, testi degli attacchi, abilità e metadati. Questa fase ha garantito un buon punto di partenza per definire il dominio e la varietà di carte disponibili.

Parallelamente, sono stati presi in esame gli archetipi di deck documentati su [Bulbapedia](#)[1], una wiki collaborativa che raccoglie descrizioni delle strategie più comuni. L'obiettivo era quello di collegare le singole carte agli archetipi, così da costurare un dataset che comprendesse non solo le carte come entità isolate, ma anche la loro aggregazione in mazzi reali.

4.1.1 Limiti riscontrati

Questo approccio ha mostrato rapidamente due criticità:

- la difficoltà di correlare in maniera univoca le carte provenienti dall'API con quelle elencate su Bulbapedia, spesso manchevoli di identificatori univoci (codice di espansione e numero di collezione, o espressi in modo diverso rispetto ai dati prelevati dall'API);
- la scarsità di esempi completi, insufficiente per addestrare un sistema supervisionato in grado di classificare e generare mazzi basati su archetipi.

Alla luce di questi limiti, l'idea di costruire un sistema basato su **apprendimento supervisionato** con mazzi etichettati per archetipo è stata accantonata.

4.1.2 Cambio di approccio

Il progetto si è quindi orientato su **apprendimento non supervisionato**, concentrandosi non più sulla classificazione dei mazzi in archetipi prestabiliti, ma sulla capacità di costruire o completare mazzi a partire da insiemi di carte. Questa scelta ha permesso di sfruttare fonti di dati più ricche e coerenti.

4.2 Dataset principali

Per supportare l'approccio non supervisionato sono stati utilizzati due dataset principali:

- La repository [pokémon-tcg-data](#)[2] dalla quale sono stati estratti:
 - mazzi di esempio per ciascuna espansione che sono stati scartati e rimossi dal dataset finale perchè non competitivi.
 - **cards by set**: liste di carte divise per espansione, con tutti i dettagli di ciascuna carta, nella cartella;
 - i metadati relativi a ciascuna espansione (nome, codice, data di rilascio, numero di carte).
- Il portale [PokéData](#)[5], dal quale sono state estratte:
 - classifiche e metadati di ogni torneo ufficiale dal 2018 in poi. Nella versione finale non sono stati utilizzati;
 - **tournament decks**: cartella contenente composizione di ciascun mazzo partecipante, con il punteggio relativo di vittorie, sconfitte e pareggi.

4.3 Struttura dei dati e selezione delle features

La struttura dei dati utilizzata nel progetto deriva principalmente dalla repository *pokémon-tcg-data*, che organizza le informazioni in tre insiemi:

- la cartella **cards by set**, con un file JSON per ogni espansione contenente tutte le carte disponibili;
- la cartella **tournament decks**, che raccoglie esempi di mazzi già utilizzabili. Il contenuto è stato poi modificato: i deck di base sono stati sostituiti con quelli prelevati da *PokéData* e suddivisi in file per ogni torneo.
- il file **sets.json**, che descrive i metadati delle espansioni.

Sono state implementate alcune classi Python per tradurre i diversi dataset in un unico formato.

4.3.1 Carte

Ogni carta è rappresentata nel dataset con una struttura JSON che contiene informazioni comuni e specifiche a seconda del **supertype**. Tutte le carte condividono alcuni campi base: l'**id**, che combina il codice del set e il numero della carta, il **name**, il **supertype** (*Pokémon*, *Trainer* o *Energy*), e il **number**, posizione della carta all'interno del set.

```
1     class Carta:
2         expansion: Expansion
3         number: str
4         name: str
5         supertype : str
6         subtypes : list[str]
```

Carte Pokémon

Le carte Pokémon sono le più complesse [Estratto 7.7]. Oltre ai dati generali, includono gli HP, il tipo, la linea evolutiva (**evolvesTo**), gli attacchi con costi ed effetti, eventuali abilità e resistenze, e il costo di ritirata.

Queste informazioni servono a definire il comportamento della carta in partita e sono state tradotte nel progetto in attributi utili al modello. Debolezze e resistenze sono state semplificate come valori booleani, così da ridurre la complessità senza perdere l'informazione principale.

Tra tutti i dati presenti sono stati ignorati quelli relativi all'estetica della carta (artista, immagini), legalità, livello, e flavor text. La classe Pokémon Card è stata implementata come segue:

```
1     class Pokémon(Carta):
2         supertype = "Pokémon"
3         type: Energy
4         evolves_from: str
5         evolves_to: list[str]
6         weaknesses: list[Energy]
7         resistances: list[Energy]
8         hp: int
9         attacks: list[Attack]
```

```
10     abilities: list[Ability]
11     retreat_cost: int

1     class Ability:
2         name: str
3         text: str
4
5     class Attack:
6         name : str
7         text : str
8         damage : int
9         cost_converted : int
10        cost : list[Energy]
```

Carte Energia

Le carte Energia hanno una struttura più snella. Si dividono in Energie Base [Listing 7.2], che forniscono un singolo tipo di energia e non hanno regole aggiuntive, ed Energie Speciali [Listing 7.1], che includono un campo **rules** con la descrizione dell'effetto speciale.

```
1     class EnergyCard(Carta):
2         supertype = "Energy"
3         rules = list[str]
```

Carte Allenatore

Le carte Allenatore si distinguono per il campo **rules**, che contiene la descrizione degli effetti e delle condizioni di utilizzo. A differenza dei Pokémon e delle energie, non hanno statistiche di gioco e tutta la loro utilità deriva dall'effetto. Ne esistono vari sottotipi, tra cui Item [Estratto 7.3], Supporter, che limitano il numero di carte giocabili per turno, e Stadium [Estratto 7.4], che restano in gioco e modificano le regole in modo permanente.

```
1     class Trainer(Carta):
2         supertype = "Trainer"
3         rules : list[str]
```

4.3.2 Deck

Nel dataset torneistico, la struttura di ogni deck è divisa in tre liste: Pokémon, Trainer e Energie, ciascuno con le carte e il numero di copie di ognuna [Estratto 7.5]. Inoltre compaiono ulteriori informazioni sui giocatori e le loro prestazioni,

come il nome, il piazzamento finale, il record di vittorie e sconfitte e, in alcuni casi, i dettagli dei turni giocati. Questo permette di collegare la composizione dei mazzi alle performance reali.

Nel progetto attuale non è implementato, ma si otterrebbero raccomandazioni ancora più adeguate attribuendo un peso ai mazzi in base al numero di vittorie.

```
1 class Deck:
2     archetipo: str
3     carte : dict[Carta, int] # carta:quantità
4     name : str
```

4.3.3 Set

I set contengono i metadati delle espansioni: nome, serie, numero di carte, codice PTCGO e data di rilascio [Estratto 7.6]. Queste informazioni sono fondamentali per normalizzare i riferimenti tra carte e deck. In origine, alcune fonti indicavano i set solo con il nome testuale, ma nel progetto i riferimenti sono stati uniformati utilizzando il PTCGO Code.

```
1 class Expansion:
2     name: str
3     code: str
4     ptcgoCode: str
5     printed : int
```

4.4 Rappresentazioni dei Deck

Nel file `structures/deck.py`, è stata implementata una classe privata `Embedding`, che definisce una serie di metodi di vettorializzazione in base agli elementi da rappresentare per ciascun mazzo. Questi vettori-mazzo possono essere concatenati per ottenere rappresentazioni più complesse e dettagliate, attraverso un processo di normalizzazione (attraverso lo `StandardScaler` di `sklearn`).

```
1 class Embedding(Enum):
2     QUANTITY = _quantity
3     KEYWORDS = _keywords
4     SUPERTYPES = _supertipi
5     SUBTYPES = _sottotipi
6     PKMN_TYPES = _energie
7     EVO_STATS = _evoluzione_stats
8     WEAKNESS = _debolezze
9     RESISTANCE = _resistenze
```

```

10     # _stat_embedding -> media, minimo, massimo e deviazione standard
11     HP = lambda m: _stat_embedding(m, lambda c: [
12         getattr(c, "hp", 0) or 0
13     ])
14     ATTACK_DMG = lambda m: _stat_embedding(m, lambda p: [
15         a.damage for a in getattr(p, "attacks", [])
16         if isinstance(a.damage, (int, float))
17     ])
18     ATTACK_COSTS = lambda m: _stat_embedding(m, lambda p: [
19         getattr(a, "cost_converted", 0)
20         for a in getattr(p, "attacks", [])
21     ] or [0]),
22
23     def __call__(self, mazzo: "Deck") -> list[float]:
24         return self(mazzo)
25
26     # ESEMPIO
27     vettore_mazzo = Deck.Embedding.QUANTITY(mazzo)

```

Embedding	Descrizione	Dominio e forma
Quantity	Copie di ciascuna carta.	$\{x \in \mathbb{N} \mid x \leq 4\}^{cu*}$
Supertypes	Distribuzione dei supertipi delle carte.	$\{x \in \mathbb{N} \mid 1 \leq x < 60\}$
Subtypes	Frequenza di sottotipi comuni nelle carte.	$\mathbb{N}^{13}$
Keywords	Frequenza di parole chiave comuni nei testi	$\mathbb{N}^{20}$
PkmnTypes	Percentuale di Pokémon per tipo di energia.	$\{x \in \mathbb{R}^+ \mid x < 1\}^{11}$
EvoStats	Percentuale di Pokémon per stadio evolutivo	$\{x \in \mathbb{R}^+ \mid x < 1\}^4$
Weakness	Percentuale di Pokémon per debolezza.	$\{x \in \mathbb{R}^+ \mid x < 1\}^{11}$
Resistance	Percentuale di Pokémon per resistenza.	$\{x \in \mathbb{R}^+ \mid x < 1\}^{11}$
HP	Statistiche sugli HP dei Pokémon.	$\mathbb{R}^+ \times \mathbb{N} \times \mathbb{N} \times \mathbb{R}^+$
AttacksDmg	Statistiche sui danni degli attacchi	$\mathbb{R}^+ \times \mathbb{N} \times \mathbb{N} \times \mathbb{R}^+$
AttacksCosts	Statistiche dei costi degli attacchi	$\mathbb{R}^+ \times \mathbb{N} \times \mathbb{N} \times \mathbb{R}^+$

Tabella 4.1: Elenco e descrizione degli embedding con relativo dominio.

cu: Carte Uniche condivise da tutti i mazzi

Capitolo 5

Knowledge Base

Questo capitolo descrive la **Knowledge Base (KB)** di *PokéVision*: sulla base del dominio rappresentato dall'ontologia definita precedentemente nel Capitolo 3, abbiamo costruito una base di conoscenza che rappresenti pienamente il dominio d'interesse, ricca di fatti e regole sui quali inferire nuova conoscenza, come la validazione di mazzi, la verifica di condizioni di gioco, offrire raccomandazioni.

Obiettivi di progettazione In questo lavoro sono state rispettate le linee guida progettuali per una buona **scalabilità** e **manutenibilità**, attraverso la separazione di modelli e vincoli strutturali (ABox) da un lato, e fatti e vincoli contestuali (TBox) dall'altro, che ci ha permesso di **allineare semanticamente** questi ultimi ai modelli definiti precedentemente nell'ontologia. Inoltre è stata resa possibile la creazione dinamica di carte e mazzi come **fatti** all'interno della KB, un controllo preventivo sui vincoli per scartare mazzi non coerenti con le regole, e conseguentemente regole di **inferenza** per ottenere conoscenza relativa alla costruzione di mazzi o alle interazioni e regole all'interno delle partite.

Classificazione di fatti nella KB Nonostante non siano classificati esplicitamente, i tipi di fatti all'interno della KB sono stati pensati in tre macro contenitori, che ci permettono di distinguere gli elementi quando all'interno e al di fuori di una partita:

- 1) **Statici di riferimento** Catalogo di carte ed espansioni.
- 2) **Derivati** Deck con le sue DeckEntry.
- 3) **Operativi** Azioni di gioco, turni, zone di gioco.

Competency Questions Esempi di domande a cui la KB può rispondere:

1. Un dato Deck, è valido?
 - (a) Ha esattamente 60 carte?
(`pkmnTCG:DeckCardCountShape`)
 - (b) Rispetta il limite di 4 carte con lo stesso nome?
(`pkmnTCG:DeckCardCopyCountShape`)
 - (c) Ha almeno 1 pokémon base?
(`pkmnTCG:hasBasicPokemon`)
 - (d) Ci sono tutte le pre-evoluzioni necessarie?
(`pkmnTCG:EvolutionChainShape`)
2. Quale Pokemon dovrei mettere in campo?
 - (a) Quali resistenze hanno i pokemon nella zona attiva avversaria?
 - (b) Quali tipi di pokemon controllo nelle mie zone?
3. Posso attivare una PlayAbilityAction?
 - (a) Ho già attivato la stessa abilità in questo turno?
(`pkmnTCG:PlayAbilityOncePerTurnShape`)
 - (b) Il pokemon è in zona di gioco?
 - (c) È il mio turno?
4. Posso effettuare un AttackAction?
 - (a) È il mio turno?
 - (b) Il pokemon in posizione attiva può pagare il costo dell'attacco?
(`pkmnTCG:AttackActionShape`)
5. È presente al massimo uno Stadium in gioco?
(`pkmnTCG:GameZoneStadiumShape`)
6. Posso evolvere un pokemon?
(`pkmnTCG:InvalidEvolvePokemonAction`)

5.1 Creazione dei Fatti

È stata gestita totalmente tramite Python, con le librerie OwlReady2 e [todo], è divisa nelle seguenti fasi sequenziali:

1. **Ingest** dati torneo (decklist, metadati mazzi). È stato effettuato il parsing del dataset torneistico, per ottenere i dati grezzi di tutte le liste dei deck
2. **Normalizzazione delle entità**. Costruite le istanze delle carte tramite `card.py`, i mazzi sono configurati come semplici contenitori.
3. **Verifica dei vincoli**. Si controlla che il grafo di un mazzo sia conforme ai vincoli imposti, altrimenti viene scartato.
4. **Trasposizione nella KB**. Tramite il metodo `to_facts()`, sia carte che mazzi sono stati trasformati in una serie di fatti che, utilizzando l'ontologia `pkmnTCG`, sono interpretabili da reasoner.

Identificazione delle istanze. Per inserire e distinguere i fatti all'interno della KB, ogni entità ha un'identificatore univoco che segue le convenzioni riassunte in Tabella 5.1.

Entità	IRI
Deck	Hashing delle combinazioni Card/quantità
Set	Codice del Set
Card	IRI del Set + numero di collezione
DeckEntry	IRI del Deck + intero progressivo
Energy	Nome dell'energia
Zone	IRI del Player + nome della zona
Abilità	IRI della carta + nome
Attacchi	IRI della carta + nome
Effetti	IRI della carta + intero progressivo

Tabella 5.1: Iri delle principali entità

5.1.1 Serializzare una Carta in RDF

Il metodo `to_facts()` della classe `Card` trasforma una carta in un fatto a partire dagli attributi Python. Di seguito un'astrazione della funzione:

```
1 def to_facts(self) -> list[str]
2     card_iri = self.get_iri()
3     fatti = [f":{card_iri} a pkmnTCG:Card, pkmnTCG:{self.supertype} ."]
4     for attributo, valore in card.attributes:
5         predicato = normalizza(attributo)
6         fatto = f":{card_iri} :{predicato} :{valore} ."
7         fatti.aggiungi(fatto)
8     return fatti
```

Esempio di output (semplificato):

```
1 :base1_4 a pkmnTCG:Card, pkmnTCG:PokemonCard .
2 :base1_4 :hasName "Charizard" .
3 :base1_4 :hasSet "base1" .
4 :base1_4 :hasCollectionNumber "4" .
5 :base1_4 :hasSubType "Stage 2" .
6 :base1_4 :hasEnergy "Fire" .
```

5.1.2 Serializzare un Deck in RDF

```
1 def to_facts(self) -> list[str]
2     deck_iri = self.iri()
3     fatti = [f':{iri} a :pkmnTCG:Deck .']
4     for card, quantity in self.carte.items():
5         entry_iri = card.get_iri() + deck_iri
6         fatti.aggiungi(f"""
7             :{entry_iri} a pkmnTCG:DeckEntry ;
8                 pkmnTCG:hasCard :{card.get_iri()} ;
9                 pkmnTCG:hasQuantity "{quantity}" .
10            {deck_iri} pkmnTCG:hasEntry :{entry_iri} .
11            """)
12     return fatti
```

Esempio per Deck e DeckEntry.

```
1 @prefix pkmnTCG: <http://example.org/pkmnTCG#> .
2 @prefix xsd:      <http://www.w3.org/2001/XMLSchema#> .
3
4 :ab12 a pkmnTCG:Deck.
5
6 :ab12-1 a pkmnTCG:DeckEntry ;
7     pkmnTCG:hasCard :SVI-123 ;
8     pkmnTCG:hasQuantity "4" .
```



```
9 :ab12 pkmnTCG:hasEntry :ab12-1 .
10
11 :ab12-2 a pkmnTCG:DeckEntry ;
12     pkmnTCG:hasCard :SVI-045 ;
13     pkmnTCG:hasQuantity "56" .
14 :ab12 pkmnTCG:hasEntry :ab12-2 .
```

5.2 Validazione dei Fatti

5.2.1 Pattern di query SPARQL

SHACL è stato l'approccio principale per la validazione diretta dei dati. I vincoli SHACL sono stati fondamentali per formalizzare le regole che richiedono restrizioni di cardinalità, confronti su valori aggregati o vincoli su proprietà annidate, logiche difficilmente esprimibili in SWRL. I vincoli sono stati implementati con SPARQLConstraints interrogando il grafo per validare le regole del gioco.

Vincoli sui Mazzi (pkmnTCG:Deck)

Questi vincoli garantiscono che ogni mazzo rispetti le regole di costruzione fondamentali del gioco.

Dimensione del Mazzo (pkmnTCG:DeckCardCountShape) Questo vincolo verifica che la somma delle quantità di tutte le carte in un mazzo sia esattamente 60. (3) indica che la regola si applica alle istanze di Deck, (10-11) tramite la query SPARQL SELECT accediamo ad ogni entryQuantity di ciascun deckEntry in Deck (13) raggruppiamo le quantità per mazzo (14) andiamo separatamente ciascun gruppo, restituendo un errore di validazione se la somma non è pari a 60.

```
1 pkmnTCG:DeckCardCountShape
2   a sh:NodeShape ;
3   sh:targetClass pkmnTCG:Deck ;
4   sh:message "Ogni mazzo deve avere esattamente 60 carte."@it ;
5   sh:sparql [
6     a sh:SPARQLConstraint ;
7     sh:select """
8       SELECT $this
9       WHERE {
10         $this pkmnTCG:hasDeckEntry ?entry .
```

```

11         ?entry pkmnTCG:entryQuantity ?qty .
12     }
13     GROUP BY $this
14     HAVING (SUM(?qty) != 60)
15     """
16 ] .

```

Limite di Copie per Carta (`pkmnTCG:DeckCardCopyCountShape`) Assicura che il numero di copie di una carta non superi il limite di 4, con l'eccezione delle energie base.

(3) anche in questo caso, il vincolo si applica a ogni istanza di `pkmnTCG:Deck`.

(10-13) tramite la query accediamo alle carte nel mazzo e al loro nome

(14-17) tramite un filtro ignoriamo le EnergyCard di sottotipo "Basic"

(19) raggruppiamo i risultati per nome della carta e per mazzo

(20) il vincolo fallisce se il totale delle copie supera il limite

```

1  pkmnTCG:DeckCardCopyCountShape
2  a sh:NodeShape ;
3  sh:targetClass pkmnTCG:Deck ;
4  sh:message "Non più di 4 copie per carta (eccetto le energie base)."@it ;
5  sh:sparql [
6      a sh:SPARQLConstraint ;
7      sh:select """
8          SELECT $this ?cardName
9          WHERE {
10             $this pkmnTCG:hasDeckEntry ?entry .
11             ?entry pkmnTCG:entryCard ?card ;
12                 pkmnTCG:entryQuantity ?qty .
13             ?card pkmnTCG:hasName ?cardName .
14             FILTER NOT EXISTS {
15                 ?card a pkmnTCG:EnergyCard ;
16                     pkmnTCG:hasSubType "Basic" .
17             }
18         }
19         GROUP BY $this ?cardName
20         HAVING (SUM(?qty) > 4)
21     """
22 ] .

```

Vincolo per la Linea Evolutiva dei Pokémon (`pkmnTCG:EvolutionChainShape`)

Assicura che un Pokémon evoluto presente nel mazzo abbia sempre la sua pre-evoluzione, garantendo la giocabilità di ogni carta.

- (3) Il vincolo si applica a ogni istanza di `pkmnTCG:Deck`.
 (8-10) La query accede alle carte del mazzo per identificare le carte evolute e i Pokémon che rappresentano.
 (11-16) Viene verificata l'esistenza della pre-evoluzione (`pkmnTCG:hasPreEvolution`) del Pokémon in esame.
 (18-21) La clausola `FILTER NOT EXISTS` verifica che all'interno dello stesso mazzo (`$this`) esista una carta che rappresenta la pre-evoluzione.

```

1  pkmnTCG:EvolutionChainShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:Deck ;
4      sh:message "Ogni Pokémon evoluto presente nel mazzo deve avere anche"
5      "la sua pre-evoluzione per garantire la giocabilità."@it ;
6      sh:sparql [
7          a sh:SPARQLConstraint ;
8          sh:select """
9              SELECT $this ?evolvedPokemon
10             WHERE {
11                 $this pkmnTCG:hasDeckEntry ?entry .
12                 ?entry pkmnTCG:entryCard ?evolvedCard .
13                 ?evolvedCard pkmnTCG:represents ?evolvedPokemon .
14
15                 ?evolvedPokemon a pkmnTCG:Pokemon ;
16                                 pkmnTCG:hasPreEvolution ?preEvolution .
17
18                 FILTER NOT EXISTS {
19                     $this pkmnTCG:hasDeckEntry ?preEntry .
20                     ?preEntry pkmnTCG:entryCard ?preCard .
21                     ?preCard pkmnTCG:represents ?preEvolution .
22                 }
23             }
24             """
25      ] .

```

Vincolo per la Linea Evolutiva dei Pokémon (`pkmnTCG:EvolutionChainShape`)

Assicura che ogni Pokémon evoluto presente nel mazzo includa tutte le sue pre-evoluzioni (Stage 1 e/o Base), garantendo la giocabilità di ogni carta.

- (3) Il vincolo si applica a ogni istanza di `pkmnTCG:Deck`.
 (8-16) La query accede alle carte del mazzo per identificare i Pokémon che rappresentano e la loro catena evolutiva.

(18-21) La clausola `FILTER NOT EXISTS` verifica che all'interno dello stesso mazzo (`$this`) esista una carta che rappresenta ogni singolo antenato trovato.

```

1  pkmnTCG:EvolutionChainShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:Deck ;
4      sh:message "Ogni Pokémon evoluto presente nel mazzo deve includere tutte le sue pre-evoluzioni"
5      sh:sparql [
6          a sh:SPARQLConstraint ;
7          sh:select """
8              SELECT $this ?evolvedPokemon ?missingAncestor
9              WHERE {
10                 # Consideriamo le carte nel mazzo corrente
11                 $this pkmnTCG:hasDeckEntry ?entry .
12                 ?entry pkmnTCG:entryCard ?card .
13                 ?card pkmnTCG:represents ?evolvedPokemon .
14                 ?evolvedPokemon a pkmnTCG:Pokemon .
15
16                 # Prendiamo TUTTI gli antenati evolutivi (uno o più passi indietro)
17                 # Se il Pokémon è Basic (senza pre-evoluzione) questa riga non matcha.
18                 ?evolvedPokemon pkmnTCG:hasPreEvolution+ ?missingAncestor .
19                 ?missingAncestor a pkmnTCG:Pokemon .
20
21                 FILTER NOT EXISTS {
22                     $this pkmnTCG:hasDeckEntry ?preEntry .
23                     ?preEntry pkmnTCG:entryCard ?preCard .
24                     ?preCard pkmnTCG:represents ?missingAncestor .
25                 }
26             }
27          """
28      ] .

```

Vincoli sulle Carte (pkmnTCG:Card)

Questi vincoli assicurano la completezza e l'accuratezza delle informazioni per ogni tipo di carta.

Effetto delle Carte Energia non Base (pkmnTCG:NonBasicEnergyCardEffectShape)

Le carte energia non di tipo Basic devono avere almeno un effetto.

```

1  pkmnTCG:NonBasicEnergyCardEffectShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:EnergyCard ;

```

```
4      sh:message "Le EnergyCard non-Basic devono avere almeno 1 effetto."@it ;
5      sh:sparql [
6          a sh:SPARQLConstraint ;
7          sh:select """
8              SELECT $this
9              WHERE {
10                 $this a pkmnTCG:EnergyCard .
11                 FILTER NOT EXISTS { $this pkmnTCG:hasSubType "Basic" }
12                 FILTER NOT EXISTS { $this pkmnTCG:hasEffect ?effect }
13             }
14             """
15      ] .
```

Vincoli Dinamici sullo Stato di Gioco (Game State)

Questi vincoli, più complessi, servono a validare le azioni e le condizioni durante una partita, non solo la struttura del mazzo.

Massimo 1 carta Stadio nella GameZone globale (pkmnTCG:GameZoneStadiumShape)
Assicura che ci sia al massimo una carta Stadio nella zona di gioco attiva.

```
1      pkmnTCG:GameZoneStadiumShape
2          a sh:NodeShape ;
3          sh:targetNode pkmnTCG:GameZone ;
4          sh:message "Ci può essere al massimo 1 carta Stadio nella GameZone."@it ;
5          sh:sparql [
6              a sh:SPARQLConstraint ;
7              sh:select """
8                  SELECT $this
9                  WHERE {
10                     $this pkmnTCG:hasCard ?c .
11                     ?c a pkmnTCG:TrainerCard ;
12                        pkmnTCG:hasSubType pkmnTCG:Stadium .
13                 }
14                 GROUP BY $this
15                 HAVING (COUNT(DISTINCT ?c) > 1)
16             """
17          ] .
```

Limiti sull'uso delle Carte per Turno (pkmnTCG:PlayCardLimitShape) Im-
pone il limite di una carta Energia, Stadio o Supporter per ogni turno.
(3) Il vincolo si applica a ogni istanza di PlayCardAction.

(8-14) Prende in considerazione le azioni compiute in un certo turno dal giocatore e la carta coinvolta.

(18-22) Classifica la carta in una delle categorie in esame.

(24-38) La sottoquery seleziona e raggruppa tutte le `PlayCardAction` dello stesso turno e dello stesso giocatore.

(40-41) Se il conteggio è superiore a 1, viene dichiarata una violazione.

```
1  pkmnTCG:PlayCardLimitShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:PlayCardAction ;
4      sh:message
5      "Massimo 1 carta di tipo Energia, Stadio o Supporter per turno e per giocatore."@it ;
6      sh:sparql [
7          a sh:SPARQLConstraint ;
8          sh:select """
9              SELECT ?this WHERE {
10                 ?this a pkmnTCG:PlayCardAction ;
11                     pkmnTCG:Turn ?Turn ;
12                     pkmnTCG:affects ?card ;
13                     pkmnTCG:performedBy ?player .
14                 ?card a pkmnTCG:Card .
15
16                 { ?card a pkmnTCG:EnergyCard . BIND("Energy" AS ?cat) }
17                 UNION
18                 { ?card a pkmnTCG:TrainerCard ; pkmnTCG:hasSubType "Stadium" .
19                   BIND("Stadium" AS ?cat) }
20                 UNION
21                 { ?card a pkmnTCG:TrainerCard ; pkmnTCG:hasSubType "Supporter" .
22                   BIND("Supporter" AS ?cat) }
23
24                 FILTER EXISTS {
25                     SELECT ?Turn ?player ?cat (COUNT(?a) AS ?n)
26                     WHERE {
27                         ?a a pkmnTCG:PlayCardAction ;
28                             pkmnTCG:Turn ?Turn ;
29                             pkmnTCG:affects ?c ;
30                             pkmnTCG:performedBy ?player .
31                         ?c a pkmnTCG:Card .
32                         { ?c a pkmnTCG:EnergyCard . BIND("Energy" AS ?cat) }
33                         UNION
34                         { ?c a pkmnTCG:TrainerCard ; pkmnTCG:hasSubType "Stadium" .
35                           BIND("Stadium" AS ?cat) }
```

```

36         UNION
37         { ?c a pkmnTCG:TrainerCard ; pkmnTCG:hasSubType "Supporter" .
38         BIND("Supporter" AS ?cat) }
39     }
40     GROUP BY ?Turn ?player ?cat
41     HAVING (?n > 1)
42 }
43 }
44 ""
45 ] .

```

Ritirata: massimo una per turno (pkmnTCG:RetreatOncePerTurnShape) As-sicura che non sia possibile ritirare un Pokémon più di una volta per turno.

```

1  pkmnTCG:RetreatOncePerTurnShape
2  a sh:NodeShape ;
3  sh:targetClass pkmnTCG:RetreatPokemonAction ;
4  sh:message "Errore: non puoi ritirare più di una volta per turno."@it ;
5  sh:sparql [
6      a sh:SPARQLConstraint ;
7      sh:select ""
8          SELECT ?turn
9          WHERE {
10             ?action a pkmnTCG:RetreatPokemonAction ;
11                 pkmnTCG:Turn ?Turn .
12          }
13          GROUP BY ?turn
14          HAVING(COUNT(?action) > 1)
15      ""
16  ] .

```

Energie non sufficienti per la ritirata (pkmnTCG:RetreatEnergyCostShape)

Controlla che il Pokémon che si ritira abbia abbastanza energie attaccate per coprire il costo.

```

1  pkmnTCG:RetreatEnergyCostShape
2  a sh:NodeShape ;
3  sh:targetClass pkmnTCG:RetreatPokemonAction ;
4  sh:message "Errore: energie non sufficienti per pagare il costo di ritirata."@it ;
5  sh:sparql [
6      a sh:SPARQLConstraint ;
7      sh:select ""
8          SELECT $this

```

```
9         WHERE {
10             $this pkmnTCG:affects ?pokemon .
11             ?pokemon pkmnTCG:hasRetreatCost ?cost .
12
13             {
14                 SELECT (COUNT(?energy) AS ?attachedEnergy)
15                 WHERE {
16                     ?pokemon pkmnTCG:hasAttachedCard ?energy .
17                     ?energy a pkmnTCG:EnergyCard .
18                 }
19             }
20
21             {
22                 SELECT (SUM(?needed) AS ?totalCost)
23                 WHERE {
24                     ?cost pkmnTCG:costQuantity ?needed .
25                 }
26             }
27             FILTER(?attachedEnergy < ?totalCost)
28         }
29         """"
30     ] .
```

PlayAbility: non usare la stessa abilità più di una volta per turno (pkmnTCG:PlayAbilityOncePerTurnShape) Impedisce l'uso ripetuto della stessa abilità nello stesso turno.

```
1  pkmnTCG:PlayAbilityOncePerTurnShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:PlayAbilityAction ;
4      sh:message "Errore: questa abilità è già stata usata in questo turno."@it ;
5      sh:sparql [
6          a sh:SPARQLConstraint ;
7          sh:select """"
8              SELECT ?Turn ?ability
9              WHERE {
10                  $this pkmnTCG:Turn ?Turn ;
11                  pkmnTCG:affects ?ability .
12              }
13              GROUP BY ?Turn ?ability
14              HAVING (COUNT($this) > 1)
15          """"
16      ] .
```


AttackAction: il Pokémon non ha abbastanza energie per usare l'attacco (pkmnTCG:AttackActionShape) Un Pokémon non può attaccare se non ha abbastanza energie assegnate per coprire il costo dell'attacco.

(10-14) la query controlla che l'azione in esame sia un attacco e che la carta che la compie sia un pokémon.

(15-28) conta quante carte energia sono assegnate al pokémon e il costo dell'attacco.

(29-31) se il numero di energie assegnate al pokémon è inferiore al costo dell'attacco viene segnalata una violazione.

```

1  pkmnTCG:AttackActionShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:AttackAction ;
4      sh:message "Il Pokémon non ha abbastanza energie per usare l'attacco."@it ;
5      sh:sparql [
6          a sh:SPARQLConstraint ;
7          sh:select """
8              SELECT $this
9              WHERE {
10                 $this pkmnTCG:affects ?pokemon ;
11                     pkmnTCG:affects ?attack .
12                 ?pokemon a pkmnTCG:PokemonCard .
13                 ?attack a pkmnTCG:Attack ;
14                     pkmnTCG:attackCost ?cost .
15                 {
16                     SELECT (COUNT(?energy) AS ?attachedEnergy)
17                     WHERE {
18                         ?pokemon pkmnTCG:hasAttachedCard ?energy .
19                         ?energy a pkmnTCG:EnergyCard .
20                     }
21                 }
22
23                 {
24                     SELECT (SUM(?needed) AS ?totalCost)
25                     WHERE {
26                         ?cost pkmnTCG:costQuantity ?needed .
27                     }
28                 }
29                 FILTER(?attachedEnergy < ?totalCost)
30             }
31             """

```

32] .

Ogni `AttackAction` deve avere almeno un bersaglio (`pkmnTCG:AttackActionTargetShape`)
Un'azione di attacco deve avere almeno un Pokémon come bersaglio.

```
1      pkmnTCG:AttackActionTargetShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:AttackAction ;
4      sh:message "Un'azione di attacco deve avere almeno un Pokémon come bersaglio."@it
5      sh:property [
6          sh:path pkmnTCG:targetsPokemon ;
7          sh:minCount 1 ;
8      ] .
```

Ogni `DiscardAction` deve specificare almeno una carta da scartare dalla `HandZone` (`pkmnTCG:DiscardActionTargetShape`) Un'azione di scarto deve avere come bersaglio una carta che si trova nella mano del giocatore.

```
1      pkmnTCG:DiscardActionTargetShape
2      a sh:NodeShape ;
3      sh:targetClass pkmnTCG:DiscardAction ;
4      sh:message
5      "Un'azione di scarto deve avere come bersaglio almeno una carta,"
6      "e tale carta deve trovarsi nella mano del giocatore (HandZone)."@it ;
7      sh:property [
8          sh:path pkmnTCG:targetsCard ;
9          sh:minCount 1 ;
10         sh:node [
11             sh:property [
12                 sh:path pkmnTCG:inZone ;
13                 sh:hasValue pkmnTCG:HandZone ;
14             ]
15         ]
16     ] .
```

5.2.2 Validazione con pySHACL

```
1  # carica grafo dati e grafo shapes
2  G = Graph().parse(PATH_DATA, format="turtle")
3  S = Graph().parse(PATH_SHAPES, format="turtle")
4  conforms, report_graph, report_text = validate(
5      data_graph=G,
6      shacl_graph=S,
7      inference='rdfs',
8      serialize_report_graph=True
9  )
10 if report_graph:
11     report_graph.serialize("/path/kb/validation_report.ttl",
12         ↪ format="turtle")
```

Listing 5.1: Validazione SHACL di un grafo RDF con pySHACL (estratto)

5.3 Regole e inferenza

5.3.1 SWRL (deduzioni strutturali)

Le regole SWRL sono state utilizzate per inferenze *discrete*, senza aggregazioni e per implementare la logica del gioco, spesso relative alla validità o meno di carte e azioni. L'output di queste regole è l'appartenenza di un individuo a una classe specifica (es. `pkmnTCG:ValidPlayAbilityAction`).

Regole sui Mazzi

Per garantire che un mazzo sia giocabile, è stato implementato un vincolo che richiede la presenza di almeno un Pokémon base. **Nota:** La classe `pkmnTCG:hasBasicPokemon`

```
pkmnTCG:Deck(?d)
~ pkmnTCG:hasDeckEntry(?d, ?e)
~ pkmnTCG:entryCard(?e, ?c)
~ pkmnTCG:PokemonCard(?c)
~ pkmnTCG:hasSubType(?c, "Basic")
-> pkmnTCG:hasBasicPokemon(?d)
```

Listing 5.2: Almeno un pokémon base per mazzo)

raggruppa tutti i mazzi che contengono almeno un Pokémon Base.

Regole sulle Carte

Per validare le proprietà delle carte in base alle loro categorie e sottotipi, sono state definite le seguenti regole SWRL. Queste regole inferiscono se una carta ha un effetto valido o se rappresenta una violazione delle regole di costruzione.

VERIFICA: Ogni `TrainerCard` deve avere almeno 1 `hasEffect`.

```
pkmnTCG:TrainerCard(?c)
~ pkmnTCG:hasEffect(?c, ?e)
-> pkmnTCG:hasValidEffect(?c)
```

Nota: La proprietà `pkmnTCG:hasValidEffect` identifica tutte le carte Allenatore che possiedono almeno un effetto valido.

VERIFICA: `EnergyCard` di tipo `Basic`: non devono avere effetti.

```
pkmnTCG:EnergyCard(?c)
~ pkmnTCG:hasSubType(?c, "Basic")
~ pkmnTCG:hasEffect(?c, ?e)
  -> pkmnTCG:InvalidBasicEnergyCard(?c)
```

Nota: La classe `pkmnTCG:InvalidBasicEnergyCard` raggruppa le carte Energia Base che violano la regola, avendo un effetto associato.

VERIFICA: Pokémon con carte attaccate solo in zone attive.

```
pkmnTCG:PokemonCard(?p)
~ pkmnTCG:hasAttachedCard(?p, ?c)
~ not(
  pkmnTCG:inZone(?p, ?z)
  ~ pkmnTCG:isInGame(?z, "true"^^xsd:boolean)
)
  -> pkmnTCG:PokemonInInvalidZone(?p)
```

Nota: La classe `pkmnTCG:PokemonInInvalidZone` rappresenta i Pokémon che hanno carte attaccate pur non trovandosi in una zona di gioco valida.

Regole sulle Azioni di Gioco

Queste regole SWRL classificano le azioni come valide o non valide in base a condizioni di gioco, come il turno corrente o le proprietà del Pokémon coinvolto.

VERIFICA: Un Pokémon non può evolversi nello stesso turno in cui è stato giocato.

```
pkmnTCG:EvolvePokemonAction(?a)
~ pkmnTCG:evolvesPokemon(?a, ?p)
~ pkmnTCG:hasTurn(?a, ?et)
~ pkmnTCG:playedOnTurn(?p, ?pt)
~ swrlb:equal(?et, ?pt)
  -> pkmnTCG:InvalidEvolvePokemonAction(?a)
```

Nota: La classe `pkmnTCG:InvalidEvolvePokemonAction` raccoglie tutte le azioni di evoluzione effettuate in un turno non consentito.

VERIFICA: PlayAbility: il Pokémon deve essere in gioco e possedere l'abilità.

```
pkmnTCG:PlayAbilityAction(?a)
~ pkmnTCG:affects(?a, ?ab)
```

```
~ pkmnTCG:hasSourcePokemon(?a, ?p)
~ pkmnTCG:hasAbility(?p, ?ab)
~ pkmnTCG:inZone(?p, ?z)
~ pkmnTCG:isInGame(?z, "true"^^xsd:boolean)
  -> pkmnTCG:ValidPlayAbilityAction(?a)
```

Nota: La classe `pkmnTCG:ValidPlayAbilityAction` rappresenta le azioni di utilizzo abilità correttamente eseguite.

VERIFICA: Le `GameAction` devono essere eseguite solo durante il proprio turno.

```
pkmnTCG:GameAction(?a)
~ pkmnTCG:hasTurn(?a, ?t)
~ pkmnTCG:performedBy(?a, ?p)
~ pkmnTCG:hasActivePlayer(?t, ?ap)
~ swrlb:notEqual(?p, ?ap)
  -> pkmnTCG:InvalidGameAction(?a)
```

Nota: La classe `pkmnTCG:InvalidGameAction` include tutte le azioni di gioco effettuate da un giocatore diverso da quello attivo.

VERIFICA: Le carte `Stadio` devono essere giocate nella `GameZone`.

```
pkmnTCG:PlayCardAction(?a)
~ pkmnTCG:affects(?a, ?c)
~ pkmnTCG:hasSubType(?c, "Stadium")
~ pkmnTCG:playsToZone(?a, ?z)
~ swrlb:notEqual(?z, pkmnTCG:GameZone)
  -> pkmnTCG:InvalidPlayStadiumAction(?a)
```

Nota: La classe `pkmnTCG:InvalidPlayStadiumAction` identifica le azioni in cui una carta `Stadio` viene giocata in una zona non valida.

5.3.2 Vincoli numerici: strategia

Esempio (pre-calcolo in Python).

```
1 Deck          = onto.search_one(iri="##Deck")
2 hasEntry      = onto.search_one(iri="##hasEntry")
3 hasQuantity   = onto.search_one(iri="##hasQuantity")
4 deckTotal     = onto.search_one(iri="##deckTotalCount")
5 for d in Deck.instances():
```

```
6         total = 0
7         for e in getattr(d, hasEntry.python_name, []):
8             q = getattr(e, hasQuantity.python_name, [])
9             total += int(q[0]) if q else 0
10        setattr(d, deckTotal.python_name, [total])
```

5.4 Esecuzione (Python, Owlready2).

```
1  ### load_swrl_rules.ipynb
2  world = World()
3  onto = world.get_ontology(ONTO_PATH).load()
4  imps = load_rules_from_json(onto, JSON_PATH)
5  print("Regole caricate: {len(imps)}")
6
7  with onto:
8      sync_reasoner_pellet(infer_property_values=True, infer_data_property_values=True)
9
10  onto.save(file="out.ttl", format="turtle")
11  print(f"Salvato out.ttl")
```

Capitolo 6

Card Recommender

In questo capitolo illustreremo come abbiamo valutato i metodi di clustering per raggruppare i mazzi in base alle loro caratteristiche, quali metriche e rappresentazioni si sono dimostrate più efficaci, e infine come abbiamo misurato questa efficacia tramite **Masked Prediction**. La valutazione di metriche, metodi e rappresentazioni è implementata nel Jupyter Notebook "3_evaluation_cluster.ipynb".

6.1 Metriche di similarità e differenza

Seguendo le rappresentazioni vettoriali già definite in `Deck.Embedding` (Cap. 4.4), possiamo definire due mazzi uguali se hanno uguale rappresentazione, o diversi in funzione alla loro divergenza, facendo appello al metodo `similarity` (Listing 6.1) in `Deck`, che permette di concatenare più embedding di due mazzi e applicarne una metrica di similarità.

Di seguito una breve panoramica delle metriche prese in considerazione in questo progetto, utili al confronto tra due mazzi:

- **Coseno**: misura l'angolo tra due vettori $\mathbf{v}_1$ e $\mathbf{v}_2$ della stessa lunghezza.

$$\cos(\theta) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\|_2 \|\mathbf{v}_2\|_2}$$

- **Overlap**: valuta la sovrapposizione normalizzata tra i due vettori, prendendo la somma dei minimi sugli stessi indici e dividendola per il massimo totale.

$$\text{sim}_{\text{overlap}}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\sum_i \min(v_{1i}, v_{2i})}{\max(\sum_i v_{1i}, \sum_i v_{2i})}$$

- **Pearson**: misura la correlazione lineare tra i due vettori, normalizzata nell'intervallo $[-1,1]$ normalizzato a $[0,1]$.

$$\text{sim}_{\text{Pearson}}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\sum_i \min(v_{1i}, v_{2i})}{\sum_i \max(v_{1i}, v_{2i})}$$


```
1 class Deck:
2     ...
3     @staticmethod
4     def similarity(
5         mazzo1 : Deck,
6         mazzo2 : Deck,
7         metrica : Similarity,
8         embedding : Embedding | list[Embedding]
9     ) -> float:
10         if not isinstance(embedding, list):
11             embedding = [embedding]
12         v1 = [x for e in embedding for x in e(mazzo1)]
13         v2 = [x for e in embedding for x in e(mazzo2)]
14         return metrica(v1, v2)
```

Listing 6.1: Metodo `Deck.similarity`, privo di normalizzazione.

6.2 Clustering dei mazzi

Per ottenere un sistema di raccomandazione non banale è fondamentale poter confrontare e raggruppare i mazzi simili tra loro. A tal fine, le rappresentazioni vettoriali descritte in `Deck.Embedding` (Cap. 4.4) sono state utilizzate come input per algoritmi di clustering non supervisionato.

Tra le varie tecniche disponibili, si è scelto di utilizzare diversi algoritmi e di valutare la qualità dei cluster ottenuti con diverse metriche.

6.2.1 Metodi di clustering

In assenza di etichette preassegnate ai dati a cui abbiamo accesso, abbiamo dovuto valutare diversi algoritmi di **clustering** per raggruppare mazzi con strutture simili in gruppi omogenei, in base alle loro proprietà latenti. Questo processo precede la reale fase di raccomandazione, riducendone il costo computazionale. Di seguito, una breve panoramica degli algoritmi utilizzati:

K-Means : assegnato il numero di cluster k , e sia T_r l'insieme di elementi campione, questo algoritmo distribuisce casualmente ogni $t \in T_r$ tra i k cluster, poi ricalcola iterativamente i centroidi dei cluster e riassegna ogni t a quello più vicino, fino a stabilizzazione.

DBSCAN - Density-Based Spatial Clustering of Applications with Noise : questo algoritmo lavora con la nozione di densità dei dati, calcolando per ogni elemento campione $t_c \in T_r$ quanti altri t si trovano entro un raggio ε , se questi superano *minPts*, viene creato un nuovo cluster centrato su t_c .

Agglomerative Clustering : questo algoritmo usa un approccio *bottom-up*, trattando ogni elemento come un cluster separato per poi unire i gruppi più vicini in base alla metrica di similarità utilizzata, fino a raggiungere una determinata distanza di threshold o un numero di cluster finali.

BIRCH - Balanced Iterative Reducing and Clustering using Hierarchies : anche questo algoritmo utilizza un approccio *bottom-up*, costruendo incrementalmente un *CF-Tree* (Cluster Feature Tree), una struttura ad albero che comprime le rappresentazioni dei dati in micro cluster di statistiche aggregate, permettendo

l'elaborazione di enormi quantità di dati, perdendo però dettagli su di essi.

Iperparametri degli algoritmi di clustering considerati

Algoritmo	Iperparametri
K-Means	Numero di cluster k
Agglomerative Clustering	Criterio di collegamento, numero di cluster (opzionale)
DBSCAN	Distanza ε , numero minimo di punti ($minPts$)
BIRCH	Soglia di espansione, fattore di ramificazione

6.2.2 Determinazione del numero ottimale di cluster

Alcuni degli algoritmi di clustering utilizzati richiedono tra gli iperparametri il numero di cluster k in cui suddividere i dati. Per fornire un valore ottimale abbiamo utilizzato l'Elbow Method e l'analisi della Silhouette. Il primo analizza la variazione dell'inerzia rappresentando una funzione a gomito per visualizzare in quale punto di essa il guadagno non aumenta più sensibilmente e fornisce in output il valore di k in quel punto. Il secondo identifica il numero di cluster che massimizza il punteggio di Silhouette, valutato su ogni $1 < k \leq 10$.

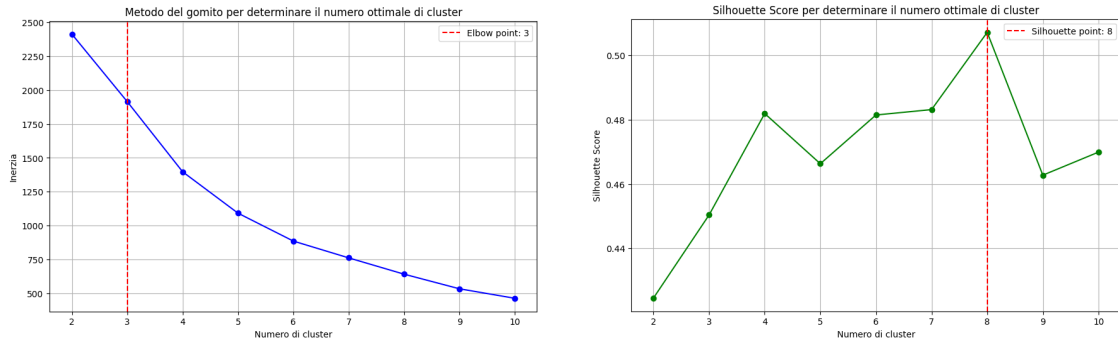


Figura 6.1: Confronto su ATTACK_COSTS, $k = 8$

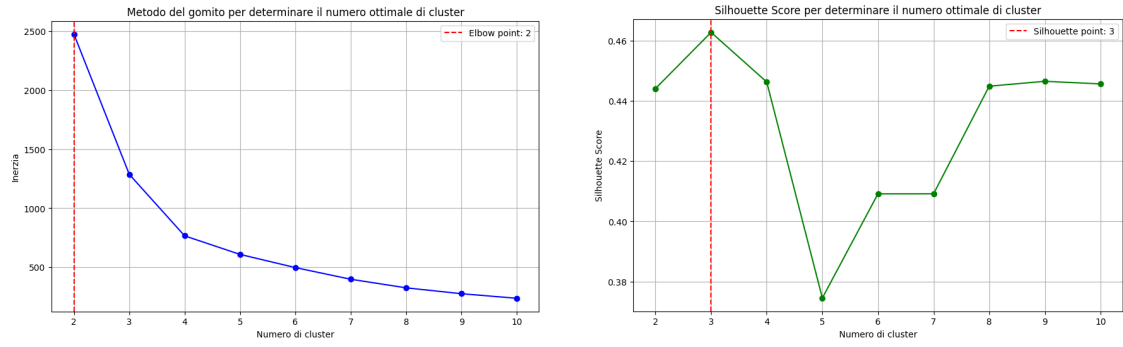


Figura 6.2: Confronto su ATTACK_DMG, $k = 3$

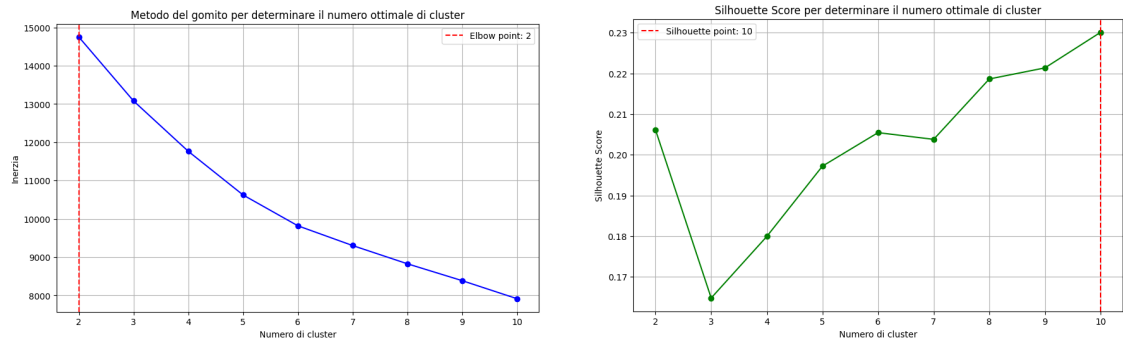


Figura 6.3: Confronto su KEYWORDS, $k = 10$

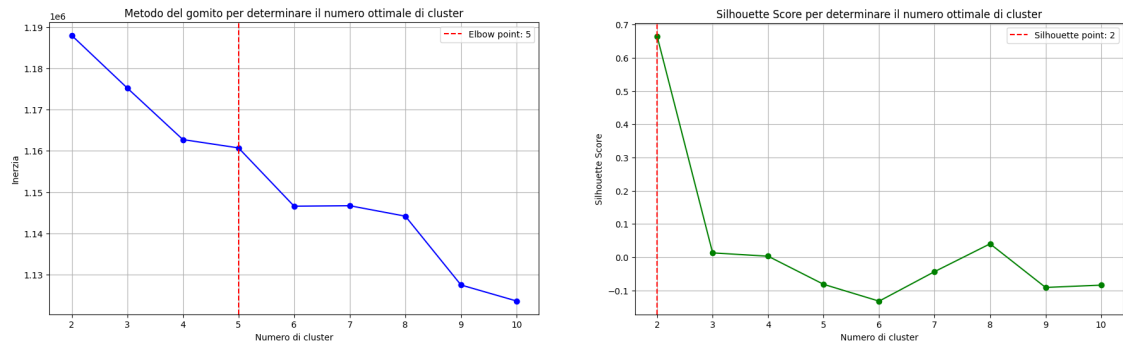


Figura 6.4: Confronto su QUANTITY, $k = 5$

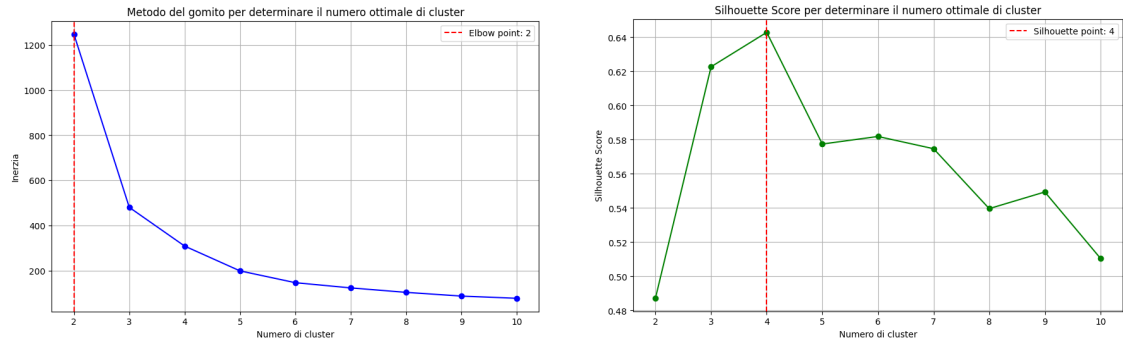


Figura 6.5: Confronto su RESISTANCE, $k = 4$

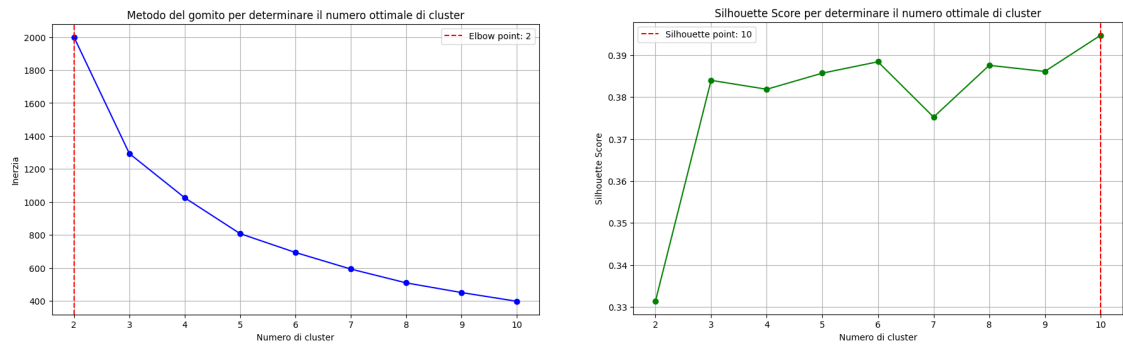


Figura 6.6: Confronto su SUPERTYPES, $k = 10$

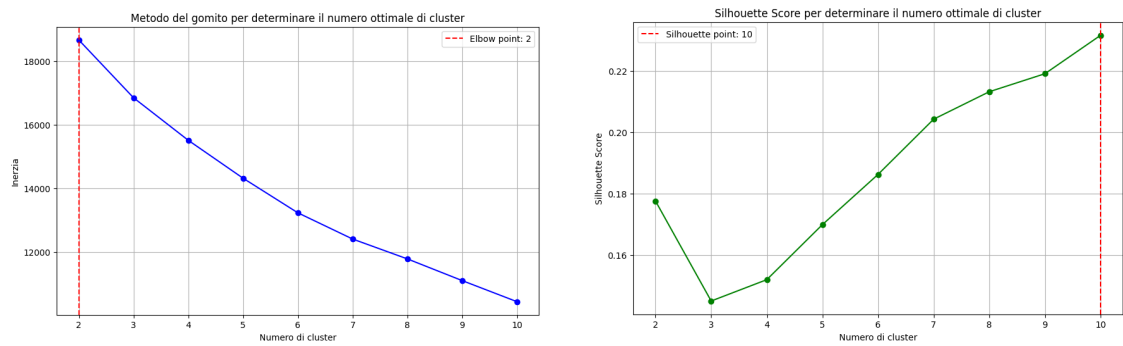


Figura 6.7: Confronto su ATTACK_COSTS + KEYWORDS, $k = 10$

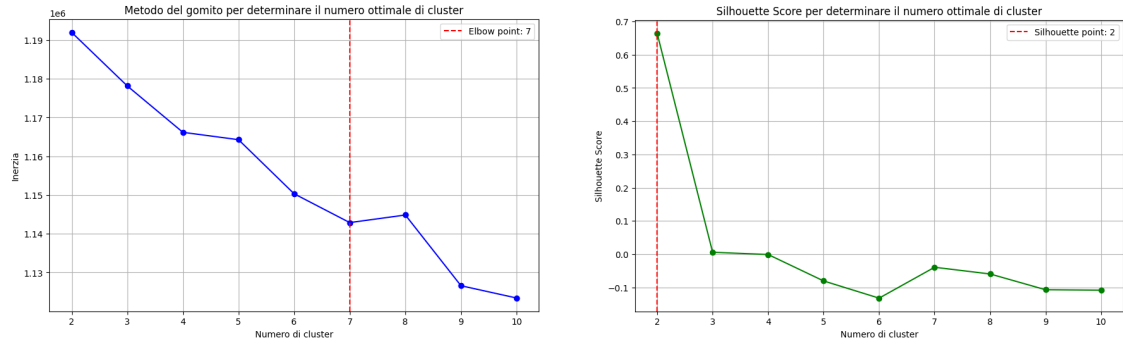


Figura 6.8: Confronto su ATTACK_COSTS + QUANTITY, $k = 7$

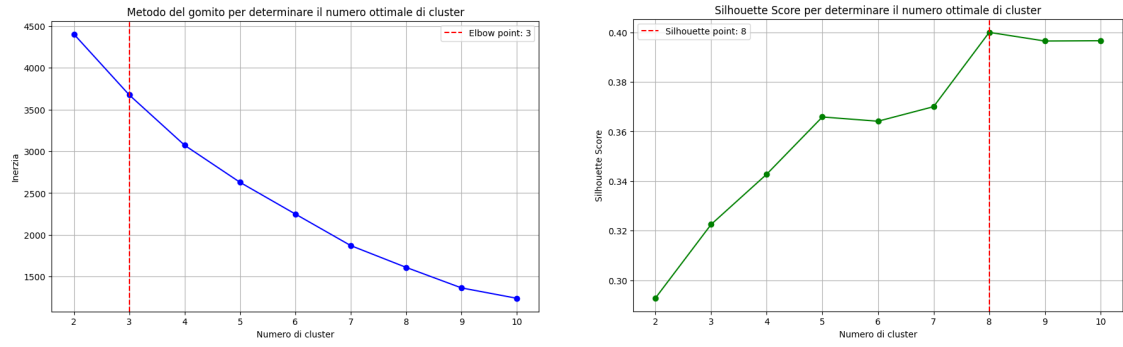


Figura 6.9: Confronto su ATTACK_COSTS + RESISTANCE, $k = 8$

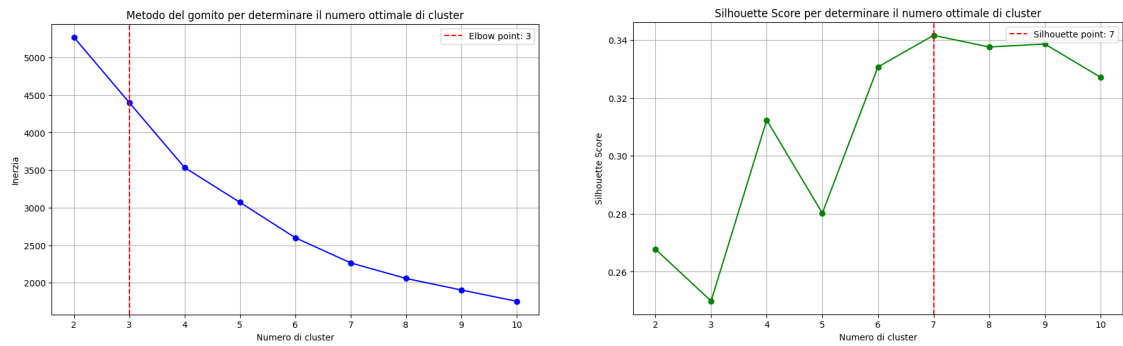


Figura 6.10: Confronto su ATTACK_COSTS + SUPERTYPES, $k = 7$

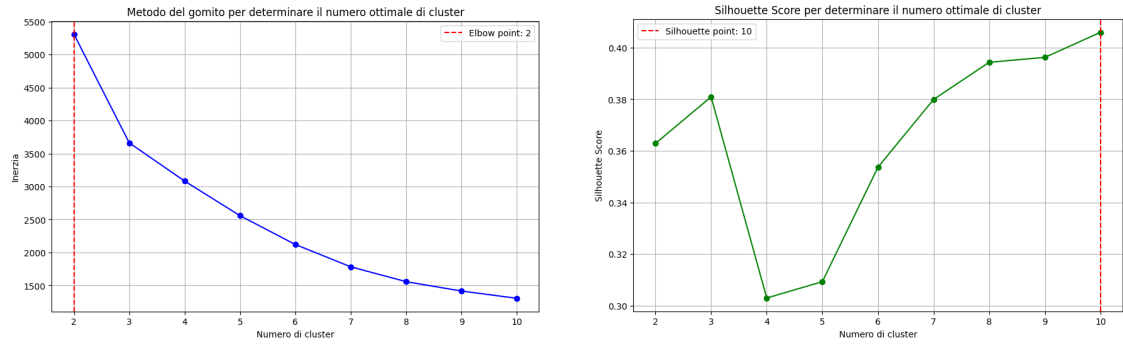


Figura 6.11: Confronto su ATTACK_DMG + ATTACK_COSTS, $k = 10$

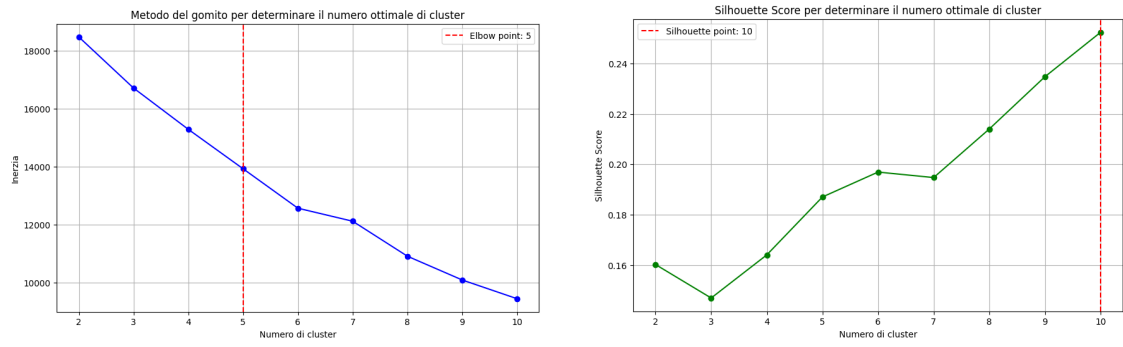


Figura 6.12: Confronto su ATTACK_DMG + KEYWORDS, $k = 10$

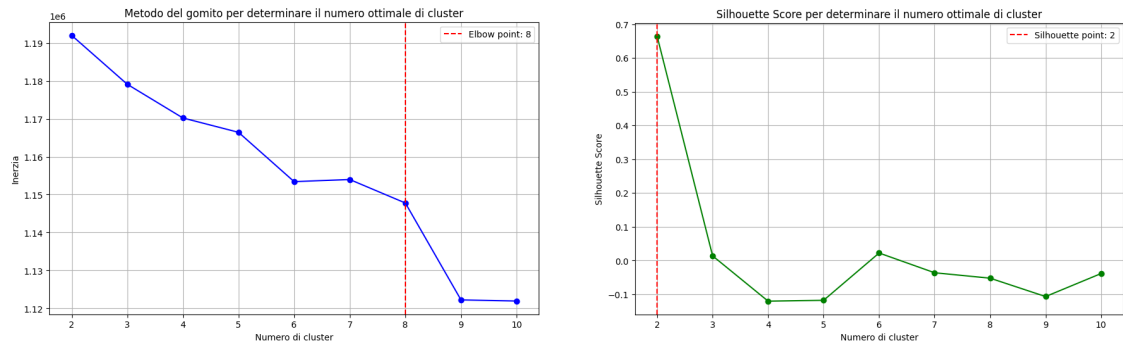


Figura 6.13: Confronto su ATTACK_DMG + QUANTITY, $k = 6$

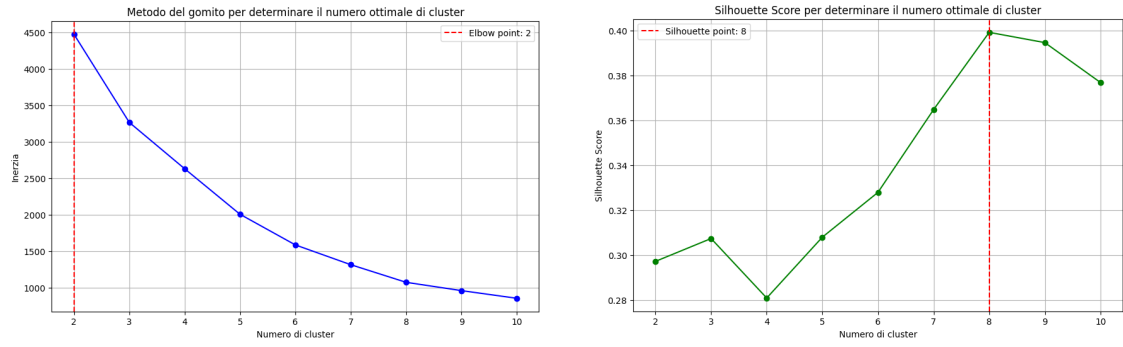


Figura 6.14: Confronto su ATTACK_DMG + RESISTANCE, $k = 8$

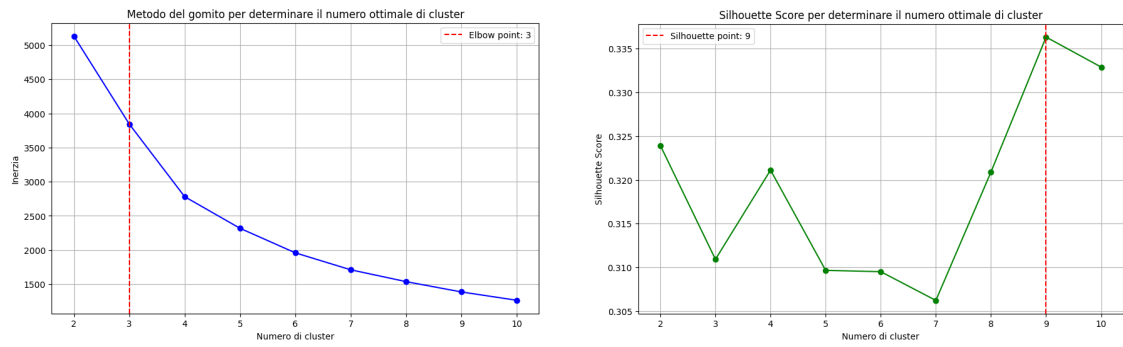


Figura 6.15: Confronto su ATTACK_DMG + SUPERTYPES, $k = 9$

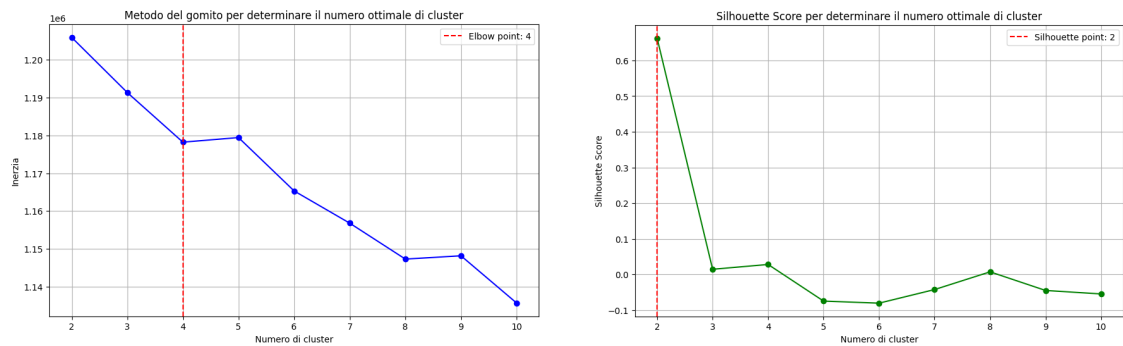


Figura 6.16: Confronto su KEYWORDS + QUANTITY, $k = 4$

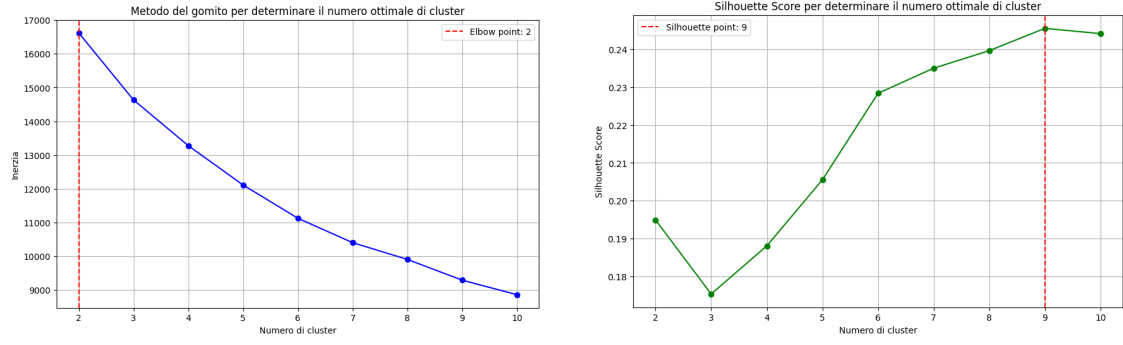


Figura 6.17: Confronto su RESISTANCE + KEYWORDS, $k = 9$

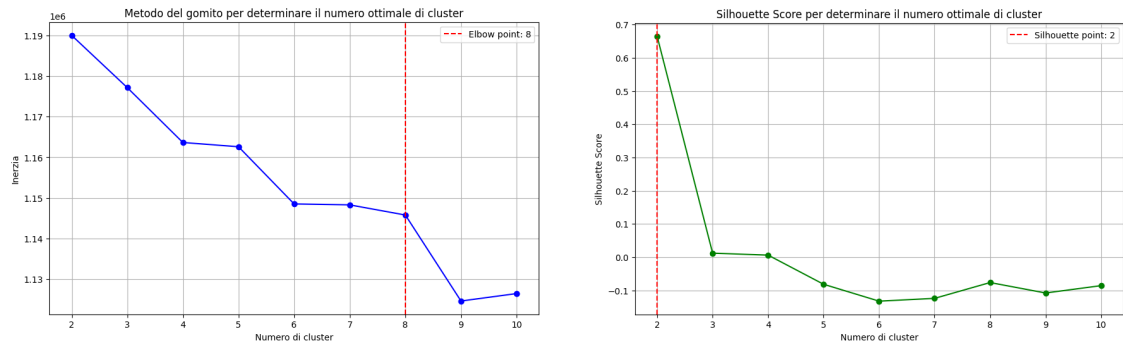


Figura 6.18: Confronto su RESISTANCE + QUANTITY, $k = 8$

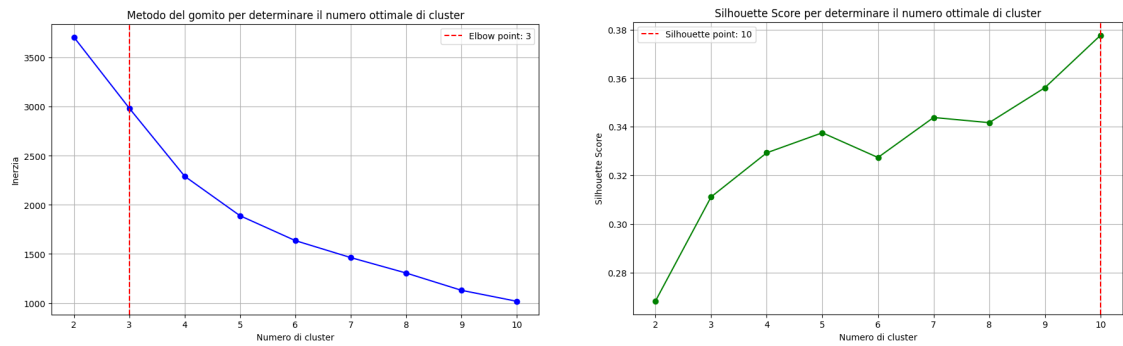


Figura 6.19: Confronto su RESISTANCE + SUPERTYPES, $k = 10$

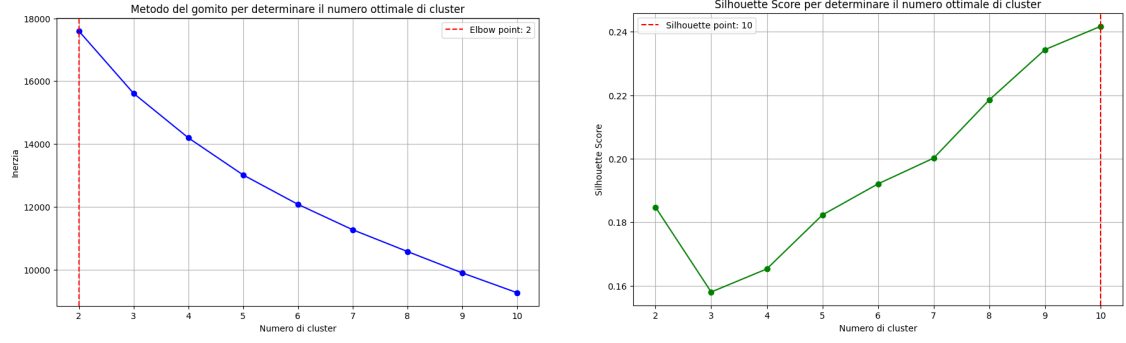


Figura 6.20: Confronto su SUPERTYPES + KEYWORDS, $k = 10$

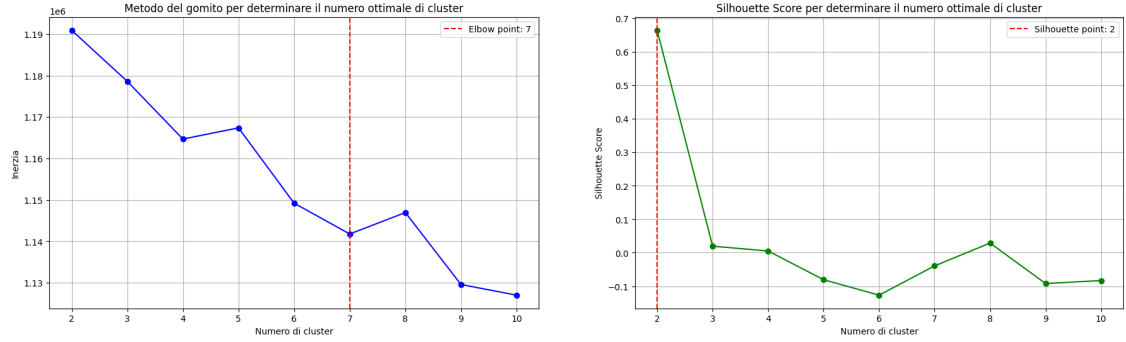


Figura 6.21: Confronto su SUPERTYPES + QUANTITY, $k = 8$

6.3 Valutazione delle rappresentazioni

Abbiamo valutato il clustering dei dati relativamente a tutti gli embedding e tutte le loro possibili combinazioni di due elementi.

Per ognuna di esse sono stati estratti i vettori di embedding per un campione di mazzi i cui dati sono stati successivamente normalizzati. Dopo aver determinato il numero di cluster ottimali per ogni algoritmo di clustering relativamente alla combinazione di embedding in esame e applicato l'algoritmo, i risultati sono stati aggregati per avere uno score complessivo del risultato.

6.3.1 Metriche di valutazione

Per valutare la migliore combinazione tra metodo di clustering, metrica di similarità e algoritmo di embedding, tra quelli precedentemente menzionati, abbiamo creato $score(i)$ una metrica basata sullo score aggregato e normalizzato delle tre metriche

più comuni, dove w_i è il peso attribuito a ciascuna di esse (è stato testato unicamente $w_i = 1$):

$$score(i) = \frac{w_1 s(i) \cdot w_2 CH(i) \cdot w_3 (-DB(i))}{w_1 + w_2 + w_3}$$

- **Silhouette Score**, che quantifica quanto un oggetto sia vicino agli elementi del proprio cluster rispetto a quelli di cluster diversi. Varia da -1 a 1, con valori attorno a 0 che indicano una separazione poco netta dagli altri cluster.
- **Calinski-Harabasz Score** che mi sura il rapporto tra la dispersione tra cluster diversi e quella intra-cluster. Più è alto il valore risultante migliore è la valutazione.
- **Davies-Bouldin Score** è la media della similarità tra ciascun cluster e il suo cluster più simile. Valori più bassi indicano una miglior separazione.

Method	k	Silh	CaHa	DaBo	exec(s)	Embedding	Similarity	Score
K-Means	4	0.6428	1817.45	0.4689	0.0116	Res	coseno	0.9595
K-Means	2	0.6651	14.3056	0.2351	1.0414	Sup_T + Q	coseno	0.6686

...

Tabella 6.1: Alcuni elementi di `results/optimal_clusters_score.csv`

6.4 Masked Prediction

Dopo aver ottenuto una suddivisione dei mazzi in cluster omogenei, il passo successivo è stato valutare la capacità del sistema di suggerire le carte mancanti attraverso un approccio di **masked prediction**. L'idea è semplice: dato un mazzo completo, si 'mascherano' alcune carte per renderlo incompleto, si determina il suo cluster di appartenenza e si sottopone al recommender system, che predice quali siano le carte più probabili da inserire, sfruttando la similarità con i mazzi dello stesso cluster.

6.4.1 Come funziona

Il processo di masked prediction è implementato e descritto nel notebook `"4_masked_prediction.ipynb"` in sintesi si articola nei seguenti passaggi:

1. **Mascheramento:** Cinque `DeckEntry` vengono rimosse da un mazzo completo scelto casualmente.

2. **Embedding e Clustering:** vengono calcolati gli embedding per il mazzo mascherato, salvati in una cache per ottimizzarne l'uso, e identificato il cluster che lo contiene mediante algoritmi di clustering.
3. **Calcolo della Similarità:** viene valutata la similarità tra l'embedding del mazzo mascherato e quello degli altri mazzi nello stesso cluster, scartando il 70% dei mazzi meno simili.
4. **Predizione delle carte:** vengono considerate le carte presenti nei mazzi più simili e non presenti in quello di test. Ad ognuna viene assegnato un punteggio in base alla frequenza e alla similarità del mazzo in cui si trovano.
5. **Ensamble di predizioni:** le predizioni fatte con diverse configurazioni di embedding e clustering vengono combinate e ponderate in base all'efficacia di ognuna. Alla fine, viene prodotta una lista ordinata per punteggio delle carte consigliate.

In questo modo, il recommender non si limita a proporre carte casuali, ma si basa sulle regolarità apprese dal clustering per ricostruire la configurazione mancante.

6.4.2 Risultati

Al momento i risultati ottenuti non sono ancora pienamente attendibili: i primi test, effettuati proponendo $y = 10$ carte e mascherandone $K = 5$, mostrano un'accuracy stimata $P(X = x)$ attorno al 63% nella predizione delle carte mascherate, utilizzando unicamente l'*Agglomerative Clustering* e come rappresentazione la concatenazione di *Keywords* e *Quantity*.

$$P(X = x \mid Y = y) = \frac{\binom{K}{x} \binom{N-K}{y-x}}{\binom{N}{y}}$$

Tali valori indicano che il sistema riesce a fornire suggerimenti migliori rispetto al caso casuale, ma restano margini significativi di miglioramento. Sono da condurre test più approfonditi con la precisione del test in funzione al numero di carte mascherate, così come altri metodi di raggruppamento e rappresentazione.

Di seguito il risultato di un test per esteso, con relativa accuracy:

Corrette	Accuracy	Predict time
3	0.6	2935.47
4	0.8	2935.47
1	0.2	2207.6
4	0.8	1869.2
3	0.6	1864.22
4	0.8	11356.36s

Accuratezza del masked prediction

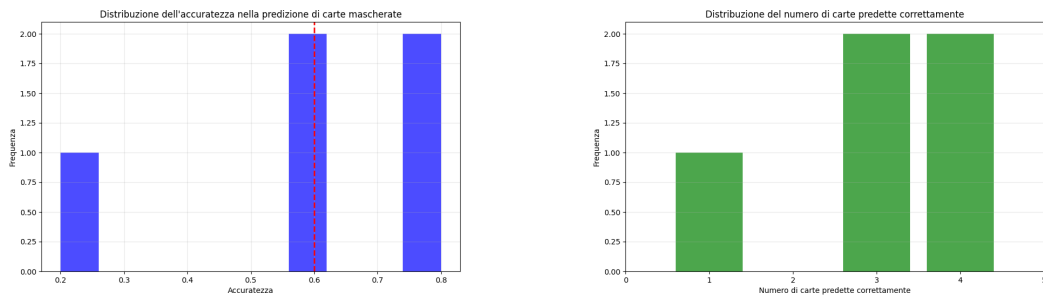


Figura 6.22

6.4.3 Vantaggi e limiti

L'approccio di masked prediction consente di testare direttamente la capacità del sistema di raccomandare carte in scenari realistici. Tra i vantaggi, spiccano la semplicità dell'implementazione e la possibilità di integrare facilmente diverse metriche di similarità. Tuttavia, i limiti sono significativi:

- l'accuracy attuale non è ancora sufficiente per un sistema di raccomandazione competitivo;
- mancano meccanismi per valutare la sinergia tra carte, non riducibili a sola frequenza o similarità.

6.4.4 Sviluppi futuri

Per migliorare le prestazioni del recommender, si considerano i seguenti sviluppi:

- ampliare la dimensione del campione
- introdurre metriche basate sulla co-occorrenza e sulla compatibilità strategica delle carte;

- valutare approcci di tipo *masked language modeling* ispirati ai modelli di deep learning.
- utilizzare metodi di soft-clustering, più appropriati per utilizzare carte condivise da più archetipi.
- sviluppo di un'interfaccia grafica per l'utilizzo del sistema con visualizzazione interattiva delle predizioni e motivazione

In conclusione, la fase di masked prediction rappresenta il primo passo concreto verso la costruzione di un **Card Recommender**, fornendo una misura pratica dell'efficacia dei cluster e aprendo la strada a miglioramenti futuri.

Capitolo 7

Estratti

```
{
  "name": "Nathan O.",
  "placing": 1,
  "record": { "wins": 7, "losses": 0, "ties": 2 },
  "decklist": {
    "pokemon": [
      {"count": 4, "name": "Sobble", "number": "41", "set": "CRE"},
      {"count": 3, "name": "Drizzile", "number": "56", "set": "SSH"}
    ],
    "trainer": [
      {"count": 4, "name": "Quick Ball", "number": "237", "set": "FST"}
    ],
    "energy": [
      {"count": 7, "name": "Darkness Energy - Basic", "set": "CRZ"},
      {"count": 3, "name": "Double Turbo Energy - Special", "set": "ASR"}
    ]
  }
}
```

Listing 7.5: Esempio di deck da torneo in formato JSON (parziale).

```
{
  "id": "base1-1",
  "name": "Alakazam",
  "supertype": "Pokémon",
  "subtypes": [
    "Stage 2"
  ],
  "level": "42",

```

```
{
  "id": "base1-96",
  "name": "Double Colorless Energy",
  "supertype": "Energy",
  "subtypes": [
    "Special"
  ],
  "rules": [
    "Provides ColorlessColorless energy.\nDoesn't count as a basic  
  ↪ Energy card."
  ],
  "number": "96",
  "artist": "Keiji Kinebuchi",
  "rarity": "Uncommon",
  "legalities": {
    "unlimited": "Legal",
    "expanded": "Legal"
  },
  "images": {
    "small": "https://images.pokemontcg.io/base1/96.png",
    "large": "https://images.pokemontcg.io/base1/96_hires.png"
  }
},
```

Listing 7.1: Esempio di Energia Speciale in formato JSON.


```
{
  "id": "base1-98",
  "name": "Fire Energy",
  "supertype": "Energy",
  "subtypes": [
    "Basic"
  ],
  "number": "98",
  "artist": "Keiji Kinebuchi",
  "legalities": {
    "unlimited": "Legal",
    "standard": "Legal",
    "expanded": "Legal"
  },
  "images": {
    "small": "https://images.pokemontcg.io/base1/98.png",
    "large": "https://images.pokemontcg.io/base1/98_hires.png"
  }
}
```

Listing 7.2: Esempio di Energia Base in formato JSON.

```
{
  "id": "bw1-92",
  "name": "Energy Retrieval",
  "supertype": "Trainer",
  "subtypes": [
    "Item"
  ],
  "rules": [
    "Put 2 basic Energy cards from your discard pile into your hand.",
    "You may play as many Item cards as you like during your turn"
    ↪ (before your attack).",
  ],
  "number": "92",
  "artist": "Kent Kanetsuna",
  "rarity": "Uncommon",
  "legalities": {
    "unlimited": "Legal",
    "standard": "Legal",
    "expanded": "Legal"
  },
  "images": {
    "small": "https://images.pokemontcg.io/bw1/92.png",
    "large": "https://images.pokemontcg.io/bw1/92_hires.png"
  }
}
```

Listing 7.3: Esempio di Trainer Item in formato JSON.

```
{
  "id": "bw4-90",
  "name": "Pokémon Center",
  "supertype": "Trainer",
  "subtypes": [
    "Stadium"
  ],
  "rules": [
    "Once during each player's turn, that player may heal 20 damage from  

    ↪ 1 of his or her Benched Pokémon.",
    "This card stays in play when you play it. Discard this card if  

    ↪ another Stadium card comes into play. If another card with the  

    ↪ same name is in play, you can't play this card."
  ],
  "number": "90",
  "artist": "5ban Graphics",
  "rarity": "Uncommon",
  "legalities": {
    "unlimited": "Legal",
    "expanded": "Legal"
  },
  "images": {
    "small": "https://images.pokemontcg.io/bw4/90.png",
    "large": "https://images.pokemontcg.io/bw4/90_hires.png"
  }
}
```

Listing 7.4: Esempio di Trainer Stadium in formato JSON.

```
[
  {
    "id": "base1",
    "name": "Base",
    "series": "Base",
    "printedTotal": 102,
    "total": 102,
    "legalities": {
      "unlimited": "Legal"
    },
    "ptcgoCode": "BS",
    "releaseDate": "1999/01/09",
    "updatedAt": "2022/10/10 15:12:00",
    "images": {
      "symbol": "https://images.pokemontcg.io/base1/symbol.png",
      "logo": "https://images.pokemontcg.io/base1/logo.png"
    }
  }
]
```

Listing 7.6: Esempio di set in formato JSON.

```
"hp": "80",
"types": [
  "Psychic"
],
"evolvesFrom": "Kadabra",
"abilities": [
  {
    "name": "Damage Swap",
    "text": "As often as you like during your turn (before your
    ↪ attack), you may move 1 damage counter from 1 of your Pokémon
    ↪ to another as long as you don't Knock Out that Pokémon. This
    ↪ power can't be used if Alakazam is Asleep, Confused, or
    ↪ Paralyzed.",
    "type": "Pokémon Power"
  }
],
"attacks": [
  {
    "name": "Confuse Ray",
    "cost": [
      "Psychic",
      "Psychic",
      "Psychic"
    ],
    "convertedEnergyCost": 3,
    "damage": "30",
    "text": "Flip a coin. If heads, the Defending Pokémon is now
    ↪ Confused."
  }
],
"weaknesses": [
  {
    "type": "Psychic",
    "value": "×2"
  }
],
"retreatCost": [
  "Colorless",
  "Colorless",
  "Colorless"
],
"convertedRetreatCost": 3,
```

```
"number": "1",
"artist": "Ken Sugimori",
"rarity": "Rare Holo",
"flavorText": "Its brain can outperform a supercomputer. Its
↳ intelligence quotient is said to be 5000.",
"nationalPokedexNumbers": [
  65
],
"legalities": {
  "unlimited": "Legal"
},
"images": {
  "small": "https://images.pokemontcg.io/base1/1.png",
  "large": "https://images.pokemontcg.io/base1/1_hires.png"
}
}
```

Listing 7.7: Esempio di carta Pokémon in JSON

Bibliografia

- [1] Bulbapedia contributors. Archetipi di mazzi del gioco di carte collezionabili pokémon.
- [2] GitHub contributors. Pokémon tcg data: Archivio github di carte, set e mazzi. Archivio non ufficiale basato sui dati forniti dall'API.
- [3] TCG Developers. Pokémon tcg api: Accesso ai dati ufficiali delle carte.
- [4] Joshua Kritz, Eduardo Mangeli, and Geraldo Xexéo. Building an ontology of boardgame mechanics based on the boardgamegeek database and the mda framework. *Game Mechanics Ontology*, 2017.
- [5] PokéData. Risultati e classifiche dei tornei pokémon tcg.